

PayrollFiti

Complete Production Build Guide Laravel + Next.js Edition

Redesigned from the original NestJS / Prisma architecture
Preserving statutory payroll correctness, structural tenant isolation, and deterministic payroll execution

September 2026

Table of Contents

PayrollFiti

Complete Production Build Guide — Laravel + Next.js Edition

Table of Contents

Part 1 — System Overview & Architecture

- 1.1 What We're Building
- 1.2 Why Rethink Instead of Translate
- 1.3 Final Architecture
- 1.4 Core Design Principles (carried forward, re-anchored in Laravel)

Part 2 — Technology Decisions & Tradeoffs

- 2.1 Technology Decision Matrix
- 2.2 "Why Not?" Sections
- 2.3 Architectural Style: Phased, Not Microservices from Day One
- 2.4 Why Sanctum: The Full Analysis
- 2.5 Why Browsershot (Headless Chromium) for Payslip PDFs, Not a Pure-PHP PDF Library
- 2.6 Why OpenTelemetry Is Introduced in Phase 2, Not Day One

Part 3 — Repository & Monorepo Setup

- 3.1 Monorepo Layout
- 3.2 Backend Scaffolding
- 3.3 Frontend Scaffolding
- 3.4 Root Orchestration

Part 4 — Docker & Local Infrastructure

- 4.1 Local Compose Topology
- 4.2 Local Development Replacements

Part 5 — PostgreSQL Database Architecture

- 5.1 Entity Overview & ERD
- 5.2 Representative Migrations
- 5.3 Indexing Strategy
- 5.4 Query Performance Discipline
- 5.5 Partitioning, Archival, and Retention
- 5.6 Backups Summary (full detail in Part 36)

Part 6 — Laravel Application Architecture

- 6.1 Why Not Controllers → Services → Models
- 6.2 Directory Structure
- 6.3 Enforcing the Boundary in CI
- 6.4 Cross-Domain Communication

Part 7 — Authentication & Authorization

- 7.1 Authentication Modes
- 7.2 Login Flow, End to End
- 7.3 Password Reset & Email Verification
- 7.4 Employee Invitations
- 7.5 Session Management (Part 17 detail; overview here)
- 7.6 RBAC: Two Independent Checks, Composed
- 7.7 Field-Level Encryption

Part 8 — Multi-Tenancy

- 8.1 Isolation Model Selection
- 8.2 Two-Layer Structural Enforcement
- 8.3 Cross-Tenant Queries (the Legitimate Exceptions)
- 8.4 Queues, Scheduled Jobs, and CLI Commands
- 8.5 Storage, Caching, and Notification Isolation

Part 9 — Payroll Calculation Engine

- 9.1 The Non-Negotiable Constraint

9.2 Namespace Layout	
9.3 Core Types	
9.4 Money & Determinism Helpers	
9.5 Proration	
9.6 The Engine Entrypoint	
9.7 The Registry — Versioned Rules by Country	
9.8 Kenya (KE) — Preserving the NHIF → SHIF Transition	
9.9 Idempotency: <code>input_hash</code> as a Structural Guarantee	
9.10 Testing the Engine	
Part 10 — Payroll API	
10.1 Execution Flow: Request → Command → Engine → Persistence → Events	
10.2 The Command	
10.3 Payroll Corrections	
10.4 Concurrency: Two Simultaneous Executions of the Same Run	
10.5 Public Payroll Calculator	
10.6 Representative Endpoint Documentation	
Part 11 — Compliance & Country Rules	
11.1 Design: Country-Gated, Template-Driven	
11.2 Adding a Fourth Country's Compliance Reports	
Part 12 — Billing & Payments	
12.1 Schema Recap & Provider Abstraction	
12.2 Never Trust a Client Redirect	
12.3 Webhook Processing	
12.4 Payment Transaction State Machine	
12.5 Reconciliation	
12.6 Automated Monthly Billing Cycle	
Part 13 — Queues, Redis & Messaging	
13.1 Why Async at All	
13.2 Queue Topology	
13.3 Retry, Backoff, and Dead-Letter Handling	
13.4 Job Uniqueness and Idempotency	
13.5 Event-Driven Fan-Out	
13.6 Payslip Generation: Job Granularity	
13.7 The Outbox Pattern — Applied Narrowly	
Part 14 — Notifications	
14.1 Provider Abstraction	
Part 15 — HR Features (Leave, Loans, Attendance, Documents)	
15.1 Leave Management	
15.2 Loans & Advances	
15.3 Attendance	
15.4 Employee Documents	
Part 16 — Employee Onboarding	
Part 17 — 2FA & Session Management	
17.1 TOTP Setup	
17.2 Sessions as Multi-Device, Individually Revocable	
Part 18 — API Keys & Webhooks	
18.1 API Keys — Scoped Machine Credentials	
18.2 Outbound Webhooks	
18.3 The Audit Log	
Part 19 — Accounting Integrations	
19.1 OAuth-Integrated Platforms	
19.2 Three Platforms, One Interface	
19.3 Wired Into Billing and Payroll	
Part 20 — Next.js Frontend Architecture	
20.1 Directory Structure	

- 20.2 Rendering Strategy per Route Type
- 20.3 Business Logic Stays in Laravel
- Part 21 — Frontend Application
 - 21.1 The API Client — Token Refresh Is Transparent (Reframed for Sanctum)
 - 21.2 Data Fetching — TanStack Query, No Hand-Rolled `useEffect`
 - 21.3 Template: Payroll Run List → Detail
 - 21.4 Employee Self-Service Portal
- Part 22 — PWA & Push Notifications
 - 22.1 Service Worker & Offline Support
 - 22.2 Web Push — Backend
 - 22.3 Install Prompt
- Part 23 — Blog & CMS
 - 23.1 Why a Headless Store, Not a Custom Admin
 - 23.2 The One Integration Point: Restricting Blog Management to One Tenant
- Part 24 — Testing Strategy
 - 24.1 The Pyramid
 - 24.2 Unit Tests (Backend)
 - 24.3 Feature/API Tests
 - 24.4 Integration Tests
 - 24.5 Contract Tests
 - 24.6 E2E Tests (Playwright)
 - 24.7 Database Testing
- Part 25 — Security Testing
- Part 26 — Load Testing
 - 26.1 k6 Scenarios
 - 26.2 Targets (documented, not invented)
- Part 27 — Observability: Structured Logging
 - 27.1 Structured JSON Logs
- Part 28 — Prometheus & Grafana
 - 28.1 Metrics Exposed
 - 28.2 Grafana Dashboards (provisioned as code, versioned in `infra/grafana/dashboards/`)
- Part 29 — Sentry & Error Tracking
- Part 30 — OpenTelemetry & Distributed Tracing
- Part 31 — Nginx
 - 31.1 Topology
- Part 32 — Production Docker
 - 32.1 Multi-Stage Dockerfile (PHP)
 - 32.2 Container-per-Process, Not Supervisor-in-Container
- Part 33 — `entrypoint.sh`
- Part 34 — CI/CD
 - 34.1 Pull Request Pipeline
 - 34.2 Merge-to-Main Pipeline
- Part 35 — Deployment & Migrations
 - 35.1 Why `php artisan migrate --force` Never Runs From Every Container
 - 35.2 Expand/Contract for Backward-Compatible Rolling Deploys
- Part 36 — Backups & Disaster Recovery
 - 36.1 PostgreSQL
 - 36.2 Object Storage
 - 36.3 Redis
 - 36.4 Scheduled Jobs (Part 6's third cross-domain communication channel)
- Part 37 — Scaling Strategy
 - 37.1 Growth Table — What Scales Linearly and What Doesn't
 - 37.2 Stateless HTTP, Statefulness Isolated to Redis/Postgres
 - 37.3 Connection Pooling, Pagination, Bulk Operations
 - 37.4 Distributed Locks, Rate Limiting, Idempotency at Scale

Part 38 — Production Hardening

38.1 Application

38.2 Infrastructure

38.3 Secrets

38.4 Environment Variable Strategy

Part 39 — Developer Runbook

39.1 First-Time Setup

39.2 Day-to-Day

39.3 Debugging a Failed Payroll Run

39.4 Adding a Fourth Country

Part 40 — Production Readiness Checklist

Complete Production Build Guide — Laravel + Next.js Edition

Provenance note. This guide regenerates *PayrollFiti — Complete Build Guide* (113 pages, NestJS/Prisma/BullMQ backend) as a Laravel/PHP backend paired with the same Next.js frontend philosophy. Every product requirement, domain boundary, and architectural invariant from the source document — statutory correctness per country, structural tenant isolation, deterministic payroll, idempotent execution — is preserved. Where this guide introduces something the original did not specify, it is explicitly labeled **Recommended Architecture (new)**. Where it changes an existing decision, it is labeled **Changed from original**, with the reason stated. Everything else is labeled implicitly by silence: unmarked content is a direct architectural carry-over, re-expressed in Laravel terms.

Table of Contents

- Part 1 — System Overview & Architecture
- Part 2 — Technology Decisions & Tradeoffs
- Part 3 — Repository & Monorepo Setup
- Part 4 — Docker & Local Infrastructure
- Part 5 — PostgreSQL Database Architecture
- Part 6 — Laravel Application Architecture
- Part 7 — Authentication & Authorization
- Part 8 — Multi-Tenancy
- Part 9 — Payroll Calculation Engine
- Part 10 — Payroll API
- Part 11 — Compliance & Country Rules
- Part 12 — Billing & Payments
- Part 13 — Queues, Redis & Messaging
- Part 14 — Notifications
- Part 15 — HR Features (Leave, Loans, Attendance, Documents)
- Part 16 — Employee Onboarding
- Part 17 — 2FA & Session Management
- Part 18 — API Keys & Webhooks
- Part 19 — Accounting Integrations
- Part 20 — Next.js Frontend Architecture
- Part 21 — Frontend Application
- Part 22 — PWA & Push Notifications
- Part 23 — Blog & CMS
- Part 24 — Testing Strategy
- Part 25 — Security Testing
- Part 26 — Load Testing
- Part 27 — Observability
- Part 28 — Prometheus & Grafana
- Part 29 — Sentry & Error Tracking
- Part 30 — OpenTelemetry & Distributed Tracing
- Part 31 — Nginx
- Part 32 — Production Docker
- Part 33 — entrypoint.sh
- Part 34 — CI/CD

- Part 35 — Deployment & Migrations
 - Part 36 — Backups & Disaster Recovery
 - Part 37 — Scaling Strategy
 - Part 38 — Production Hardening
 - Part 39 — Developer Runbook
 - Part 40 — Production Readiness Checklist
-

Part 1 — System Overview & Architecture

1.1 What We're Building

PayrollFiti is a multi-tenant SaaS platform that runs payroll and manages HR data for companies across Kenya, Nigeria, and South Africa. Each tenant (a customer organization) has one or more companies, employees, salary structures, and its own subscription and billing history — fully isolated from every other tenant sharing the same database.

Three constraints make this system non-trivial, and they are unchanged by the framework rewrite:

1. **Statutory correctness per country.** Kenya, Nigeria, and South Africa each run their own income-tax bands, social-security schemes, and levies, and those rules change over time (Kenya replaced NHIF with SHIF on 1 Oct 2024). The system must produce the right numbers today *and* be able to reproduce a payslip generated under last year's rules, forever.
2. **Tenant isolation.** One PostgreSQL database serves every customer. A bug that lets Tenant A see Tenant B's employees is the worst possible failure for this product category, so isolation must be structural — enforced by the framework, not by every engineer remembering `WHERE tenant_id = ?`.
3. **Deterministic money.** The same inputs must always produce the same payslip, forever, even after code changes, because a payslip is a legal and financial record.

Everything in this guide — module boundaries, queueing, database design, testing — exists in service of those three constraints. Laravel does not weaken any of them; it changes *how* they are enforced, and in a few places (tenancy, queues, the ORM) it changes the enforcement mechanism for the better. Those changes are called out explicitly throughout.

1.2 Why Rethink Instead of Translate

A mechanical NestJS → Laravel port would map `@Injectable()` services onto `app/Services`, Prisma models onto Eloquent models 1:1, and BullMQ queues onto Laravel queues, and stop there. That produces working code that fights the framework: Eloquent used like Prisma loses its most useful features (relationships, observers, query scopes), and a services-only architecture in Laravel — unlike NestJS, which forces module boundaries through its DI container — tends to erode into an unstructured pile, because nothing in vanilla Laravel stops a controller from reaching into any model in the system.

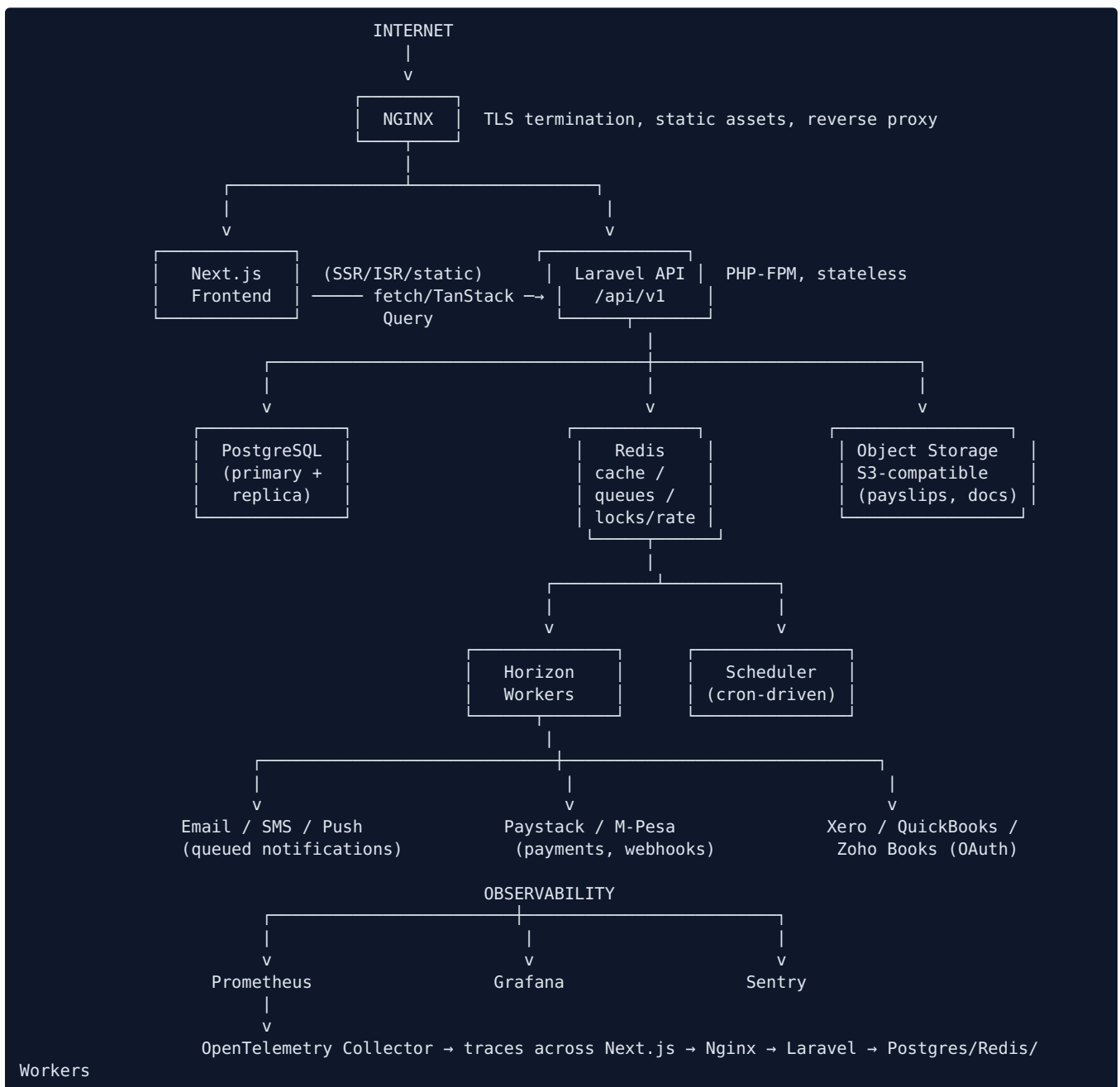
This guide instead asks, for every mechanism: *what problem did this solve in the original architecture, and what is the idiomatic, structurally-enforced Laravel equivalent?* The answers reshape several major decisions:

Original mechanism	Problem it solved	Laravel redesign
Prisma client extension reading <code>AsyncLocalStorage</code>	Structural tenant scoping	Global Eloquent scope bound from tenant-resolution middleware, backed by Postgres Row-Level Security as a second, DB-level line of defense (Part 8)
NestJS module system (<code>@Module</code>)	Domain boundary enforcement via DI	Explicit <code>Domain/<Context></code> folders, each a self-contained Laravel package registered via its own <code>ServiceProvider</code> — DI boundaries enforced by PHP namespace + Composer PSR-4 autoload discipline, not just convention (Part 6)
BullMQ (Redis-backed) queues	Async payslip/notification/webhook work	Laravel Queues on Redis, orchestrated with Horizon; same queue-per-concern topology, plus job uniqueness and batching primitives BullMQ required hand-rolling (Part 13)
<code>packages/payroll-rules</code> — dependency-free TS package	Pure, testable payroll core	A Composer-autoloaded PHP namespace (<code>Domain\Payroll\Engine</code>) with zero framework imports — same guarantee,

Original mechanism	Problem it solved	Laravel redesign
		enforced by a dependency-direction test in CI rather than physical package isolation (Part 9)
Dual guards (RolesGuard + PermissionsGuard)	Coarse + fine-grained authorization	Laravel Gates for coarse role checks + Policies for fine-grained, per-model permission checks, composed the same “both must pass” way (Part 7)

The result is not “PayrollFiti but the backend is PHP.” It is PayrollFiti’s product requirements re-derived inside a different framework’s grain, changing implementation mechanics where Laravel’s idioms are genuinely stronger (declarative authorization, Eloquent’s relationship graph, Horizon’s operational visibility) and preserving mechanics where the original’s design was framework-agnostic and already correct (the pure calculation core, versioned rule resolution, idempotency-by-input-hash).

1.3 Final Architecture



1.4 Core Design Principles (carried forward, re-anchored in Laravel)

Pure calculation core. `Domain\Payroll\Engine\PayrollCalculator::calculate(PayrollInput $input, CountryRuleSet $rules): PayrollResult` has no side effects: no `now()`, no Eloquent, no I/O, no randomness. Same input + same rule version → same output, byte for byte. This is what lets the system hash `(input, ruleVersion)` into a deterministic `input_hash` and use it as an idempotency key (Part 9).

Versioned country rules, resolved by effective date. Every country's rules are a list of `CountryRuleSet` implementations, each declaring an `effectiveFrom` date. Resolution always takes an explicit date — the payroll period being processed, never `now()` — so a March 2024 payroll run recomputed in 2026 still uses March 2024 rules. New legislation ships as a new class, never a mutation of an old one (Part 9, Part 11).

Structural multi-tenancy. Enforced at two independent layers: a global Eloquent scope resolved from request-bound tenant context (application layer), and PostgreSQL Row-Level Security policies (database layer) as defense against any code path — a raw query, a forgotten scope, a queue job with stale context — that bypasses the first layer (Part 8).

Two-layer authorization. Laravel Gates enforce coarse role checks (`admin`, `hr`, `employee`); Policies enforce fine-grained, per-resource permission checks (`payroll.run`, `employees.write`). Both are consulted; policy checks run through `Gate::authorize()` calls that Form Requests invoke declaratively, so authorization logic cannot silently leak out of the declared boundary into ad hoc controller `if` statements (Part 7).

The rest of this guide builds the system in dependency order: infrastructure → database → domain core (payroll engine) → application layer (API, tenancy, auth) → integrations (payments, notifications, accounting) → frontend → testing → deployment → operations.

Part 2 — Technology Decisions & Tradeoffs

2.1 Technology Decision Matrix

Area	Selected	Alternatives Considered	Why Selected	Tradeoff Accepted
Backend framework	Laravel 11 (PHP 8.3+)	NestJS (original), Symfony	Explicit brief requirement; mature ecosystem for queues, auth, ORM, testing all first-party and cohesive	Smaller pool of engineers fluent in <i>both</i> Laravel and TypeScript than a TS-only stack; PHP's request-per-process model needs deliberate design for long-lived state (Horizon workers)
Frontend framework	Next.js 15 (App Router)	Remix, plain SPA (Vite + React)	Unchanged from original — route groups separate marketing/app, SSR/ISR fit the public marketing site + blog, client components fit the authenticated dashboard	Two deploy targets (Next.js + Laravel) instead of one; requires a defined API contract instead of shared TS types across a single language boundary
Database	PostgreSQL 16	MySQL 8	Unchanged from original — native <code>CHECK</code> constraints, partial/expression indexes, <code>JSONB</code> with indexing, native Row-Level Security (needed for tenancy defense-in-depth, §8)	None material; Postgres is the stronger default for this workload class
ORM	Eloquent	Prisma-style raw query builder only, Doctrine	First-party, deeply integrated with Laravel's auth, validation, queue serialization, and testing (model factories); global scopes give a clean seam for tenant enforcement	Eloquent's "convenience" (mass assignment, magic relations) is a footgun without discipline — mitigated with strict <code>\$fillable</code> , typed casts, and Larastan static analysis in CI
Cache / Queue / Locks	Redis	RabbitMQ (queue only), Memcached (cache only)	One piece of infrastructure serving cache, queues, rate limiting, and distributed locks, with first-class Laravel integration (Horizon, <code>Cache::lock()</code> , <code>RateLimiter</code>)	Redis becomes critical shared infrastructure — a Redis outage degrades queues, cache, <i>and</i> rate limiting simultaneously (mitigated in §37 with Redis Sentinel/cluster and documented degraded-mode behavior)
Queue monitoring	Laravel Horizon	Custom dashboard, Bull Board equivalent	Purpose-built for Redis queues, live metrics, per-queue throughput/failure visibility, config-as-code	Horizon itself needs a supervised, always-running process — one more thing to keep alive (systemd/Supervisor in Docker, §33)
Auth mechanism	Laravel Sanctum (token abilities) for the public API; session cookies for the first-party Next.js app	Passport (full OAuth2 server), stateless JWT	The system has two client types with different needs — see §2.4 for the full "Why Sanctum" analysis	Sanctum's token abilities are coarser than a full OAuth2 scope system; acceptable because PayrollFiti is not acting as a third-party OAuth

Area	Selected	Alternatives Considered	Why Selected	Tradeoff Accepted
				provider (it <i>consumes</i> OAuth for accounting integrations, Part 19, which is unrelated to inbound auth)
API style	REST, versioned at / api/v1	GraphQL	Unchanged from original — the domain has well-defined, mostly non-nested resources (employees, payroll runs, payslips); REST's cacheability and simplicity outweigh GraphQL's flexible-querying benefit here	Some dashboard views need 2–3 sequential requests instead of one GraphQL query — mitigated with purpose-built “read model” endpoints for the heaviest dashboard views (§10.6) rather than a general query layer
PDF generation	Laravel + spatie/browsershot (headless Chromium) rendering Blade templates , with <code>barryvdh/laravel-dompdf</code> as a fallback for simple documents	<code>@react-pdf/renderer</code> (original)	The original rendered payslips as React components so branding was “just props.” In Laravel, payslip templates are Blade views (still just props via view data) rendered to PDF; Browsershot gives pixel-accurate CSS (flexbox/grid) matching what a React-rendered PDF could do — see §2.5	Browsershot requires a headless Chromium binary in the worker image (heavier image, slower cold start) — isolated to payslip/report-generation queue workers only, not the API container (§32)
Background jobs	Laravel Queues (Redis driver) + Horizon	Symfony Messenger-style, custom cron polling	Native, same Redis instance already in use, first-class retry/backoff/uniqueness support	None material
Payments	Paystack + M-Pesa (Daraja) , behind a <code>PaymentProvider</code> interface	Direct coupling to one provider	Unchanged from original — both are African-market-first, together cover cards/bank (Paystack) and mobile money (Daraja)	Two provider integrations to maintain, two sets of webhook edge cases — the interface abstraction contains the blast radius (§12)
Object storage	S3-compatible (AWS S3 in prod, MinIO in dev)	Local disk only	Payslips and employee documents cannot live only on one container's ephemeral disk if the API scales horizontally	Requires signed-URL plumbing and a small amount of extra config vs. <code>Storage::disk('local')</code> — Laravel's filesystem abstraction makes the two nearly interchangeable in code (§22)
Metrics	Prometheus	Datadog, CloudWatch	Open, self-hostable, pull-based scraping fits a Laravel app exposing a <code>/metrics</code> endpoint cheaply	Self-managed retention/storage/HA (mitigated: Prometheus + long-term remote-write is a well-trodden path, and this is a “when you need it” addition, not day-one infra, §27)
Dashboards	Grafana	Kibana, CloudWatch dashboards	Pairs natively with Prometheus; one pane of glass across app, infra, and business (payroll/payments) metrics	None material

Area	Selected	Alternatives Considered	Why Selected	Tradeoff Accepted
Error tracking	Sentry (unchanged from original, DSN-gated, off by default locally)	Rollbar, Bugsnag	First-class Laravel <i>and</i> Next.js SDKs, release tracking, performance tracing in one product	Cost scales with event volume — mitigated with sampling and fingerprint-based rate limiting on noisy exception types
Tracing	OpenTelemetry	Vendor-proprietary APM agent	Vendor-neutral; exports to Prometheus/Grafana Tempo or a commercial backend without re-instrumenting	Added later than day one — see §2.6 “why not immediately”
E2E testing	Playwright (unchanged)	Cypress	Already the right tool independent of backend language; multi-browser, strong Next.js support	None material
Load testing	k6	JMeter, Locust	Scriptable in JS (matches frontend team’s language), first-class Prometheus output for correlating load-test runs with backend metrics	None material
Security scanning	Dependabot + Semgrep + Trivy + Gitleaks + OWASP ZAP	Single all-in-one commercial scanner	Each tool covers a distinct layer (dependency CVEs, SAST, container CVEs, secret leakage, DAST) with no license cost, composable in CI	More CI configuration surface than one vendor tool — justified because no single free tool covers all five layers well (§25)
Containers	Docker	Podman	Universal tooling support, what every managed container platform expects	None material
Reverse proxy	Nginx	Traefik, Caddy	Battle-tested for exactly this shape (static Next.js + PHP-FPM upstream), explicit config over auto-discovery magic — matches an ops team that wants to <i>read</i> the routing rules (§31)	Traefik’s auto-service-discovery would reduce config for a container-orchestrated deployment; acceptable tradeoff at the “modular monolith on a handful of hosts” scale this guide targets (§37)

2.2 “Why Not?” Sections

Why Laravel instead of NestJS?

NestJS’s strength is that its module/DI system *forces* domain boundaries — you cannot easily reach into another module’s providers without importing them. Laravel has no equivalent enforcement out of the box; this guide compensates with an explicit `Domain/<Context>` folder structure (Part 6), each with its own `ServiceProvider`, and a Deptrac or custom static-analysis rule in CI that fails the build if `Domain\Payroll` imports from `Domain\Billing` directly instead of through a declared contract. In exchange, Laravel gives first-class, low-ceremony answers for auth (Sanctum), authorization (Policies/Gates), queues (Horizon), and testing (model factories, HTTP test assertions) that NestJS either doesn’t ship or requires more assembly to get. For a team standardizing on PHP, or integrating with an existing PHP estate, this is a net win; for a greenfield team with no PHP experience, the original NestJS stack remains a reasonable choice — this guide’s brief was specifically to redesign around Laravel.

Why PostgreSQL instead of MySQL?

Unchanged from the original: native partial indexes, `CHECK` constraints for statutory-value invariants, `JSONB` with GIN indexing for flexible compliance-report payloads, and — new to this redesign — native Row-Level Security, which becomes the second enforcement layer for tenant isolation (§8.2). MySQL 8 has improved on several of these but Postgres remains the stronger default for this workload.

Why Redis instead of RabbitMQ?

Laravel's queue system, Horizon, cache, rate limiter, and distributed-lock helpers are all built with Redis as the primary target. RabbitMQ (or SQS) would work as a queue driver but would require running a second piece of infrastructure just for cache and rate limiting, with no corresponding benefit for this system's throughput profile (queue depths in the thousands, not millions, per run). The cost is that Redis becomes a single point of failure across four concerns at once — addressed with Redis Sentinel for HA and explicit degraded-mode behavior per concern (§15.6, §37).

Why Laravel Horizon?

Horizon is Redis-queue-specific, so it only makes sense given the Redis decision above — but given that decision, it is close to a strict upgrade over a custom dashboard: per-queue throughput, wait time, and failed-job browsing out of the box, plus declarative supervisor configuration (worker counts, balancing strategy) that would otherwise need to be hand-rolled and monitored separately.

Why a modular monolith instead of microservices?

The original guide explicitly rejected premature service extraction, and nothing about the Laravel rewrite changes that calculus — if anything, Laravel's ecosystem makes a well-organized monolith even more viable than a hand-rolled NestJS one, because the framework already gives you queues, cache, and auth as in-process concerns instead of things you'd be tempted to peel off into services just to get them out of a tangled codebase. See §2.3 and Part 37 for the explicit phase plan and extraction triggers.

Why REST instead of GraphQL?

PayrollFiti's resources are not deeply nested or highly variable per client — a payroll run detail page needs the same shape whether requested from the web dashboard or a future mobile client. REST's cacheability (ETags, `Cache-Control`), simpler authorization story (permission checks per-route rather than per-field), and lower operational surface (no N+1-prone resolver graph to reason about) outweigh GraphQL's main benefit — avoiding over/under-fetching — which is mitigated instead with purpose-built aggregate endpoints for the few screens that would otherwise need several round trips (§10.6).

Why Sanctum instead of Passport or stateless JWT?

See §2.4.

Why Prometheus + Grafana?

Open-source, self-hostable, and the de facto standard for pull-based metrics from a PHP-FPM + queue-worker topology; Grafana's dashboard-as-code (provisioned JSON) keeps dashboards in version control alongside the application, rather than configured by hand in a vendor UI.

Why Sentry?

Exception tracking and Prometheus-style metrics answer different questions: Prometheus tells you *the error rate on* `POST /payroll/runs` spiked at 14:02; Sentry tells you *why* — the actual stack trace, the request payload (redacted), the release that introduced it, and how many distinct users hit it. Both are needed; neither replaces the other (§29).

Why OpenTelemetry — and why not on day one?

See §2.6.

Why Docker?

Universal, matches what every managed platform (ECS, Cloud Run, a bare VM with Docker Compose) expects, and gives local dev and production the same base images — eliminating “works on my machine” drift for PHP extension versions, which matter for a payroll system doing exact-decimal arithmetic (Part 9).

Why Nginx?

The topology here — static-optimized Next.js in front, PHP-FPM behind — is Nginx’s textbook use case, and explicit `location` blocks make routing rules auditable by a security reviewer without learning a new proxy’s configuration model. Traefik’s auto-discovery is a better fit for a fast-changing container-orchestrated fleet; PayrollFiti’s Phase 1–2 topology (Part 37) doesn’t need that.

Why S3-compatible object storage?

Payslips and employee documents are legal records that must survive container restarts and be retrievable regardless of which API instance served the original request — ruling out local disk once the API runs on more than one host. Laravel’s filesystem abstraction (`Storage::disk('s3')`) makes this a config change, not a code change, versus local dev.

Why an Outbox pattern (for payroll completion and payment settlement only)?

See Part 13 §13.7. Applied narrowly, not universally — most events in this system tolerate an at-least-once, eventually-consistent queue dispatch; the two exceptions (payroll-run completion, payment settlement) are where a lost event has financial or compliance consequences.

Why not Kafka?

Kafka solves problems this system does not have: multiple independent consumer groups replaying the same event stream at different rates, retention-based event sourcing, or sustained throughput in the tens of thousands of events per second. PayrollFiti’s peak async workload — payslip generation and delivery for a payroll run — is bounded by tenant size (thousands, not millions, of jobs per run) and fits comfortably on Redis-backed queues with room to spare; introducing Kafka here would add an operational burden (a distributed log to run, monitor, and reason about) with no corresponding requirement to justify it.

2.3 Architectural Style: Phased, Not Microservices from Day One

```
Phase 1 – Modular Laravel Monolith
|
+---- PostgreSQL (single primary)
+---- Redis (single instance)
+---- Horizon queue workers (in-process, same codebase)
+---- Next.js (single deployment)
+---- Nginx (single reverse proxy)

Phase 2 – Horizontally Scaled Monolith
|
+---- Multiple stateless Laravel API instances behind a load balancer
+---- Multiple Horizon worker fleets, scaled independently of API instances
+---- Redis Sentinel (HA) or managed Redis cluster
+---- PostgreSQL primary + read replica(s)
+---- Object storage (S3) for all file output

Phase 3 – Extract Only Proven Bottlenecks
|
+---- Payroll processing service (if payroll-run CPU/DB load starts starving API request latency)
+---- Notification/delivery service (if notification volume outgrows a shared worker fleet)
+---- Reporting/export service (if large exports cause memory pressure on shared workers)
```

Signals that justify moving Phase 1 → Phase 2 (all observable, not speculative): p95 API latency under normal load exceeds the documented SLO (§26) on a single instance at current traffic; Horizon queue wait time for the `payroll` queue regularly exceeds job SLA during peak (end-of-month) processing; a single Postgres primary’s CPU or connection count is the binding constraint under load testing.

Signals that justify moving Phase 2 → Phase 3 (per component, not all-or-nothing): the payroll-processing workload's resource profile (CPU-bound, bursty at month-end) is measurably starving the notification or API workloads sharing the same worker fleet, *and* horizontally adding more of the shared fleet no longer helps because the contention is for a specific bottleneck (e.g., a lock, a specific table's write throughput) rather than raw capacity. Extraction is justified by a measured, specific bottleneck — never performed speculatively “because it might scale better later.” Each extracted service takes its bounded context's tables and queues with it; nothing is extracted until Part 6's domain boundaries make that a mechanical move rather than a redesign.

2.4 Why Sanctum: The Full Analysis

PayrollFiti has two distinct client types:

1. **The first-party Next.js dashboard** — same-origin-adjacent (or configured as a trusted origin), needs session-like behavior: silent renewal, CSRF protection, no token ever visible to JavaScript.
2. **Third-party API consumers** — the public API described in Part 18, authenticated with long-lived API keys/tokens, no browser, no CSRF concern, needs coarse-grained scoping (“this key can read employees but not run payroll”).

Option	Fits client type 1 (dashboard)?	Fits client type 2 (public API)?	Verdict
Sanctum, cookie-based SPA auth for the dashboard + Sanctum personal access tokens for the public API	Yes — <code>httpOnly</code> cookie, automatic CSRF via <code>EnsureFrontendRequestsAreStateful</code> , no token exposed to JS	Yes — <code>abilities()</code> give exactly the coarse per-key scoping the public API needs	Selected. One package, two auth modes, both first-class in Laravel
Laravel Passport (full OAuth2 server)	Overkill — no third-party app is doing an OAuth authorization-code dance to access <i>its own</i> dashboard	Overkill for a straightforward API-key model; better fit if PayrollFiti needed to let <i>external developers'</i> end users grant delegated access (it doesn't — API keys belong to the tenant itself, Part 18)	Rejected — solves a “many third-party apps acting on behalf of many end users” problem PayrollFiti doesn't have
Stateless JWT (custom)	Workable but reimplements what Sanctum's cookie mode already does, and now needs manual short-lived-access + refresh-token rotation, plus a revocation story (JWTs can't be revoked without a denylist, which reintroduces state anyway)	Workable, but loses Sanctum's per-token <code>abilities()</code> and <code>last_used_at</code> visibility without extra code	Rejected — no material benefit over Sanctum for either client type, more custom code to secure correctly

Session/refresh strategy for the dashboard: Sanctum's SPA mode uses Laravel's normal session cookie (`httpOnly`, `Secure`, `SameSite=Lax` for the first-party origin), backed by the `sessions` table (or Redis, §15.5) for horizontal scaling. There is no separate “refresh token” concept to manage — the session simply persists (with configurable idle timeout, Part 7 §7.5), which is simpler and *more* secure than the original's manual JWT-access + JWT-refresh pair, because there is no refresh token to steal from `localStorage` or leak through an XSS vector. The original guide's own §Part 13 dedicated real engineering effort to closing exactly that class of gap (multi-tab session behavior, silent refresh races); Sanctum's cookie model removes the problem class rather than patching it.

2.5 Why Browsershot (Headless Chromium) for Payslip PDFs, Not a Pure-PHP PDF Library

The original used `@react-pdf/renderer` specifically so payslip branding (logo, accent color) was “just props” on a React component, with real flexbox layout. `dompdf` (pure-PHP) supports only a subgrid of CSS — no flexbox, no CSS Grid — which would force a step backward in layout fidelity for a legal document that includes tenant-branded headers, itemized earning/deduction tables, and multi-language content (§11 covers per-country statutory report formats, which have their own strict layout requirements). Browsershot renders a normal Blade view in headless Chromium and exports a PDF, giving the same CSS capability the original had. The cost is a heavier worker image and slower per-document render (~200–400ms with a warm Chromium instance vs. ~30–60ms for `dompdf`); this is why PDF generation is isolated to dedicated queue workers (Part 21) rather than run inline on the API request thread, and why

the `reports` queue (simpler, single-page compliance reports) uses `dompdf` while the `payslips` queue (branded, tabular) uses Browsershot — matching tool to document complexity rather than defaulting to the heaviest option everywhere.

2.6 Why OpenTelemetry Is Introduced in Phase 2, Not Day One

Tracing pays for itself once requests fan out across multiple hops with independent failure/latency profiles — Next.js → Nginx → Laravel → Postgres/Redis → queue worker → external provider. In Phase 1 (single API instance, single worker fleet, synchronous debugging via Laravel's own request logs and Telescope in local/staging), the *marginal* value of distributed tracing over structured logs with correlation IDs (§27) is low relative to its instrumentation and collector-operation cost. It is introduced at the Phase 1 → Phase 2 boundary (§2.3), specifically when horizontal scaling makes “which of these five API instances handled the slow request, and where did the time actually go” a question structured logs alone answer poorly.

Part 3 — Repository & Monorepo Setup

3.1 Monorepo Layout

The original's Turborepo/pnpm monorepo shape is preserved — a shared root housing a Next.js app and a backend app — but the backend is now a Laravel application, and the “shared pure package” that mattered most (`payroll-rules`) becomes a namespace *inside* the Laravel app rather than a separately published package, because PHP has no equivalent to a workspace-linked TS package consumed by two different runtimes (Next.js never needs to import PHP code directly — it only ever talks to the payroll engine through the HTTP API, including the public payroll calculator, §10.5, which becomes a public unauthenticated endpoint rather than an in-browser calculation).

```
payrollfiti/
├── apps/
│   └── web/                                # Next.js 15 App Router frontend
│       ├── app/
│       │   ├── (marketing)/
│       │   └── (app)/
│       ├── components/
│       ├── lib/
│       │   ├── api/                       # typed API client, one module per backend domain
│       │   └── validation/                # Zod schemas mirroring Laravel Form Requests
│       ├── hooks/
│       ├── features/                      # feature-oriented: payroll/, employees/, leave/, loans/, billing/
│       └── tests/
├── backend/
│   └── laravel/
│       ├── app/
│       │   ├── Domain/                   # bounded contexts – see Part 6
│       │   ├── Application/              # commands, queries, DTOs – see Part 6
│       │   ├── Infrastructure/           # persistence, external services – see Part 6
│       │   └── Http/                     # controllers, middleware, requests, resources
│       ├── database/
│       │   ├── migrations/
│       │   ├── factories/
│       │   └── seeders/
│       ├── routes/
│       │   └── api.php
│       ├── tests/
│       │   ├── Unit/
│       │   ├── Feature/
│       │   └── Integration/
│       └── config/
├── docker/
│   ├── nginx/
│   ├── php/
│   ├── supervisor/
│   └── entrypoint.sh
├── infra/
│   ├── prometheus/
│   └── grafana/
├── .github/
│   └── workflows/
├── docker-compose.yml
├── docker-compose.dev.yml
├── docker-compose.prod.yml
└── README.md
```

There is deliberately no root-level `packages/` directory shared between `apps/web` and `backend/laravel` — a PHP codebase and a TypeScript codebase cannot share source files, only a *contract*. That contract is an OpenAPI specification generated from the Laravel routes (§43), from which the frontend generates its TypeScript types. This is the one place the redesign removes something the original had (a literally shared package) and replaces it with a generated equivalent — flagged here because it's a structural consequence of the language boundary, not a stylistic preference.

3.2 Backend Scaffolding

```
mkdir payrollfiti && cd payrollfiti
composer create-project laravel/laravel backend/laravel "^11.0"
cd backend/laravel
composer require laravel/sanctum laravel/horizon laravel/scout \
    spatie/laravel-permission spatie/browsershot barryvdh/laravel-dompdf \
    league/flysystem-aws-s3-v3 sentry/sentry-laravel \
    laravel/telescope --dev
composer require pestphp/pest pestphp/pest-plugin-laravel larastan/larastan --dev
php artisan install:api    # publishes Sanctum config + migration
```

`spatie/laravel-permission` is used only for its role/permission *storage and query* primitives (`roles` , `permissions` , `model_has_permissions` tables) — the actual **enforcement** is Laravel's native Gates/Policies wired on top of it (Part 7), not the package's own blade directives or middleware, to keep authorization decisions in one declarative place per resource rather than scattered across route middleware strings.

3.3 Frontend Scaffolding

```
cd ../../apps/web
pnpm create next-app@latest . --typescript --tailwind --app --turbopack
pnpm dlx shadcn@latest init
pnpm add @tanstack/react-query zod react-hook-form @hookform/resolvers
pnpm add -D vitest @testing-library/react @playwright/test
```

3.4 Root Orchestration

Root `package.json` (frontend-only scripts; backend scripts run through Composer/Artisan directly since there is no cross-language task runner needed once the shared-package rationale for Turborepo is gone):

```
{
  "name": "payrollfiti",
  "private": true,
  "workspaces": ["apps/*"],
  "scripts": {
    "web:dev": "pnpm --filter web dev",
    "web:build": "pnpm --filter web build",
    "web:test": "pnpm --filter web test",
    "web:test:e2e": "pnpm --filter web test:e2e"
  }
}
```

`Makefile` at the repo root gives a single entry point spanning both languages for local development and CI parity:

```
.PHONY: up down migrate seed test-backend test-frontend fmt
```

```
up:  
  docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d
```

```
down:  
  docker compose down
```

```
migrate:  
  docker compose exec api php artisan migrate
```

```
seed:  
  docker compose exec api php artisan db:seed
```

```
test-backend:  
  docker compose exec api php artisan test --parallel
```

```
test-frontend:  
  pnpm --filter web test
```

```
fmt:  
  docker compose exec api ./vendor/bin/pint  
  pnpm --filter web lint --fix
```

Part 4 — Docker & Local Infrastructure

4.1 Local Compose Topology

Local development mirrors production's process separation from day one (no combined “run everything in one container” shortcut), because payroll-processing failure modes (queue starvation, PHP-FPM worker exhaustion) only show up when workers and the API are genuinely separate processes.

```

# docker-compose.yml — base services shared by dev and prod overrides
name: payrollfiti

services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: ${DB_USERNAME:-payrollfiti}
      POSTGRES_PASSWORD: ${DB_PASSWORD:-payrollfiti}
      POSTGRES_DB: ${DB_DATABASE:-payrollfiti}
    volumes: ["db-data:/var/lib/postgresql/data"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USERNAME:-payrollfiti}"]
      interval: 5s
      timeout: 5s
      retries: 10

  redis:
    image: redis:7-alpine
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 5s
      retries: 10

  api:
    build: { context: ./backend/laravel, dockerfile: ../../docker/php/Dockerfile, target: app }
    env_file: ./backend/laravel/.env
    depends_on:
      db: { condition: service_healthy }
      redis: { condition: service_healthy }
    healthcheck:
      test: ["CMD", "php", "artisan", "health:check"]
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 20s

  horizon:
    build: { context: ./backend/laravel, dockerfile: ../../docker/php/Dockerfile, target: app }
    command: php artisan horizon
    env_file: ./backend/laravel/.env
    depends_on:
      db: { condition: service_healthy }
      redis: { condition: service_healthy }
    healthcheck:
      test: ["CMD-SHELL", "php artisan horizon:status | grep -q running"]
      interval: 15s
      timeout: 5s
      retries: 6

  scheduler:
    build: { context: ./backend/laravel, dockerfile: ../../docker/php/Dockerfile, target: app }
    command: sh -c "while true; do php artisan schedule:run >> /dev/stdout 2>&1; sleep 60; done"
    env_file: ./backend/laravel/.env
    depends_on: { db: { condition: service_healthy } }

  nginx:
    build: { context: ./docker/nginx }
    ports: ["8080:80"]
    depends_on: { api: { condition: service_healthy } }

  web:
    build: { context: ./apps/web, dockerfile: ../../docker/web/Dockerfile }
    environment:
      NEXT_PUBLIC_API_URL: ${NEXT_PUBLIC_API_URL:-http://localhost:8080}
    depends_on: { api: { condition: service_healthy } }
    healthcheck:
      test: ["CMD-SHELL", "node -e \"require('http').get('http://localhost:3000/', r => process.exit(r.statusCode<500?0:1)).on('error',
()=>process.exit(1))\\\""]
      interval: 10s

```

```

timeout: 5s
retries: 10
start_period: 15s

```

```

volumes:
  db-data:

```

Architectural Trade-offs

Decision	Chosen Approach	Alternative	Why Chosen	Cost/Tradeoff
Health check for <code>api</code>	<code>php artisan health:check</code> custom Artisan command	Shell out to <code>curl localhost/health</code>	Base PHP image doesn't ship <code>curl</code> ; an Artisan command reuses the same readiness logic exposed at <code>/health</code> (§25) without adding a package	One more Artisan command to maintain, but it's ~10 lines and shares code with the HTTP health endpoint
Scheduler process	Polling loop calling <code>schedule:run</code> every 60s	<code>cron</code> inside the container	No system cron daemon needed inside a minimal container image; identical behavior, one less installed package, easier to healthcheck as a normal process	Slight imprecision (jobs scheduled to the exact minute may run up to a few seconds late) — irrelevant for this system's actual schedule granularity (§36)
<code>horizon</code> as separate service from <code>api</code>	Two containers, one image	Run Horizon as a background process inside the API container via Supervisor	Independent scaling and independent healthchecks/restarts — an API deploy shouldn't restart in-flight payroll jobs, and a worker crash shouldn't take the API down	One more service to define/monitor — accepted because it directly enables Part 37's independent-scaling requirement

4.2 Local Development Replacements

Production dependency	Local dev replacement	Why
SES/SMTP email provider	Mailpit (<code>axllent/mailpit</code>)	Catches all outbound mail in a local web UI; zero risk of a dev/test run emailing a real address
S3 object storage	MinIO	S3-API-compatible, runs in a container, exercises the exact <code>s3</code> filesystem driver code path used in production
Sentry	Disabled (<code>SENTRY_DSN=</code> empty)	Matches the original's "off by default, DSN-gated" behavior — local exceptions go to <code>storage/logs/laravel.log</code> and Telescope instead
Prometheus/Grafana	Included in <code>docker-compose.dev.yml</code> but optional (<code>--profile observability</code>)	Full observability stack isn't needed for day-to-day feature work; available on demand for anyone actively working on Part 27–29

```
# docker-compose.dev.yml — additive overrides + dev-only services
services:
  api:
    volumes: ["/backend/laravel:/var/www/html:cached"]
    command: php artisan serve --host=0.0.0.0 --port=8000
    environment:
      APP_DEBUG: "true"

  mailpit:
    image: axllent/mailpit
    ports: ["8025:8025"]

  minio:
    image: minio/minio
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: payrollfiti
      MINIO_ROOT_PASSWORD: payrollfiti-local
    ports: ["9000:9000", "9001:9001"]
    volumes: ["minio-data:/data"]

  prometheus:
    image: prom/prometheus
    profiles: ["observability"]
    volumes: ["/infra/prometheus:/etc/prometheus"]
    ports: ["9090:9090"]

  grafana:
    image: grafana/grafana
    profiles: ["observability"]
    ports: ["3002:3000"]

volumes:
  minio-data:
```

Local quickstart (Part 39 has the full runbook):

```
git clone <repo> && cd payrollfiti
cp backend/laravel/.env.example backend/laravel/.env
cp apps/web/.env.example apps/web/.env.local
docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d
docker compose exec api php artisan key:generate
docker compose exec api php artisan migrate --seed
```

Part 5 — PostgreSQL Database Architecture

5.1 Entity Overview & ERD

The schema's entities are unchanged from the original — this is a functional-baseline preservation area (§59): every table below existed as a Prisma model in the source document. What changes is the enforcement mechanism (Eloquent + migrations + RLS instead of Prisma + client extension) and a handful of column-level improvements Postgres/Eloquent make natural (e.g., `numeric` with explicit precision for every money column, enforced at the schema level, not just the application level).

```
erDiagram
    TENANTS ||--o{ COMPANIES : owns
    TENANTS ||--o{ USERS : has
    TENANTS ||--|| SUBSCRIPTIONS : has
    COMPANIES ||--o{ EMPLOYEES : employs
    EMPLOYEES ||--o{ CONTRACTS : has
    EMPLOYEES ||--o{ ONBOARDING_TASKS : has
    EMPLOYEES ||--o{ SALARY_STRUCTURES : assigned
    SALARY_STRUCTURES ||--o{ SALARY_STRUCTURE_COMPONENTS : contains
    SALARY_COMPONENTS ||--o{ SALARY_STRUCTURE_COMPONENTS : referenced_by
    COMPANIES ||--o{ PAYROLL_RUNS : runs
    PAYROLL_RUNS ||--o{ PAYROLL_ENTRIES : contains
    EMPLOYEES ||--o{ PAYROLL_ENTRIES : receives
    PAYROLL_ENTRIES ||--o{ PAYSLEIPS : generates
    EMPLOYEES ||--o{ LEAVE_REQUESTS : submits
    EMPLOYEES ||--o{ LEAVE_BALANCES : has
    EMPLOYEES ||--o{ LOANS : holds
    LOANS ||--o{ LOAN_REPAYMENTS : scheduled
    EMPLOYEES ||--o{ ATTENDANCE_RECORDS : logs
    EMPLOYEES ||--o{ EMPLOYEE_DOCUMENTS : uploads
    TENANTS ||--|| SUBSCRIPTIONS : subscribes
    SUBSCRIPTIONS }o--|| PLANS : selects
    TENANTS ||--o{ INVOICES : billed
    INVOICES ||--o{ PAYMENT_TRANSACTIONS : paid_by
    TENANTS ||--o{ API_KEYS : issues
    TENANTS ||--o{ WEBHOOK_ENDPOINTS : registers
    WEBHOOK_ENDPOINTS ||--o{ WEBHOOK_DELIVERY_LOGS : records
    TENANTS ||--o{ AUDIT_LOGS : records
    USERS ||--o{ AUDIT_LOGS : performs
```

5.2 Representative Migrations

Every table carries `tenant_id` (except `tenants` itself and global reference tables like `plans`), a design choice explained in full in Part 8. Money columns are `numeric(14,2)` — never `float/double` — matching the original's determinism requirement (Part 9).

```
// database/migrations/2025_01_01_000001_create_tenants_table.php
Schema::create('tenants', function (Blueprint $table) {
    $table->uuid('id')->primary();
    $table->string('name');
    $table->string('slug')->unique();
    $table->string('subdomain')->unique()->nullable();
    $table->jsonb('branding')->nullable(); // logo_url, accent_color — used by payslip/report templates
    $table->timestampTz('trial_ends_at')->nullable();
    $table->timestampsTz();
});

// database/migrations/2025_01_01_000010_create_employees_table.php
Schema::create('employees', function (Blueprint $table) {
    $table->uuid('id')->primary();
    $table->foreignUuid('tenant_id')->constrained()->cascadeOnDelete();
    $table->foreignUuid('company_id')->constrained()->cascadeOnDelete();
    $table->string('employee_number');
    $table->string('first_name');
    $table->string('last_name');
    $table->string('email')->nullable();
    $table->string('national_id_encrypted')->nullable(); // field-level encryption, §7.7
    $table->string('country', 2); // KE, NG, ZA — drives rule-set resolution, Part 9
    $table->date('hire_date');
    $table->date('termination_date')->nullable();
    $table->string('status')->default('active'); // active | on_leave | terminated
    $table->foreignUuid('salary_structure_id')->nullable()->constrained();
    $table->timestampsTz();
    $table->softDeletesTz(); // justified: terminated employees must remain in historical payroll joins

    $table->unique(['tenant_id', 'company_id', 'employee_number']);
    $table->index(['tenant_id', 'company_id', 'status']);
});

// database/migrations/2025_01_01_000030_create_payroll_runs_table.php
Schema::create('payroll_runs', function (Blueprint $table) {
    $table->uuid('id')->primary();
    $table->foreignUuid('tenant_id')->constrained()->cascadeOnDelete();
    $table->foreignUuid('company_id')->constrained()->cascadeOnDelete();
    $table->date('period_start');
    $table->date('period_end');
    $table->date('pay_date');
    $table->string('status')->default('draft'); // draft | processing | completed | failed | approved | corrected
    $table->char('input_hash', 64); // sha256 of the canonical PayrollInput — idempotency key, Part 9
    $table->string('rule_version'); // e.g. "KE-2025", frozen at run time for reproducibility
    $table->foreignUuid('initiated_by')->constrained('users');
    $table->foreignUuid('approved_by')->nullable()->constrained('users');
    $table->timestampTz('approved_at')->nullable();
    $table->foreignUuid('corrects_run_id')->nullable()->constrained('payroll_runs');
    $table->timestampsTz();

    $table->unique(['tenant_id', 'company_id', 'input_hash']); // structural idempotency, Part 9 §9.6
    $table->index(['tenant_id', 'company_id', 'period_start', 'period_end']);
    $table->check('period_end >= period_start');
});

// database/migrations/2025_01_01_000031_create_payroll_entries_table.php
Schema::create('payroll_entries', function (Blueprint $table) {
    $table->uuid('id')->primary();
    $table->foreignUuid('tenant_id')->constrained()->cascadeOnDelete();
    $table->foreignUuid('payroll_run_id')->constrained()->cascadeOnDelete();
    $table->foreignUuid('employee_id')->constrained();
    $table->decimal('gross_pay', 14, 2);
    $table->decimal('net_pay', 14, 2);
    $table->decimal('total_deductions', 14, 2);
    $table->jsonb('breakdown'); // immutable snapshot of every line item — earnings, deductions, tax bands applied
    $table->timestampTz('created_at');
    // No updated_at: payroll entries are immutable once written — corrections create a new
    // payroll_run with corrects_run_id set, never an UPDATE on a historical entry (§9.8).

    $table->unique(['payroll_run_id', 'employee_id']);
});
```

```
$table->index(['tenant_id', 'employee_id']);
});
```

Full `CHECK` constraint coverage matters for a financial schema: `gross_pay >= 0`, `net_pay >= 0`, `total_deductions >= 0`, `net_pay = gross_pay - total_deductions` (the last enforced by a `CHECK` rather than trusted-to-application-code, so a bug anywhere in the write path cannot silently corrupt a payslip's arithmetic at rest).

5.3 Indexing Strategy

Every foreign key gets an index (Postgres does not auto-index FKs, unlike the primary key side). Beyond that, indexes are added for the query patterns that actually run, not speculatively:

Table	Index	Query it serves
payroll_runs	(tenant_id, company_id, input_hash) unique	Idempotency check on run submission (Part 9 §9.6)
employees	(tenant_id, company_id, status)	Active-employee listing, the single most frequent dashboard query
payroll_entries	(tenant_id, employee_id)	Employee's payslip history
audit_logs	(tenant_id, created_at DESC)	Recent-activity feed, always time-ordered and tenant-scoped
webhook_delivery_logs	(webhook_endpoint_id, created_at DESC)	Delivery-history pagination, Part 18
notifications	(tenant_id, user_id, read_at) partial (WHERE read_at IS NULL)	Unread-count badge — the hottest read path in the notification bell, so it gets a partial index rather than a full one, keeping the index small as read notifications accumulate

5.4 Query Performance Discipline

Every Eloquent relationship that appears in a listing endpoint's `Resource` transformation must be eager-loaded (`with()/load()`), enforced by `Model::preventLazyLoading(! app()->isProduction())` in `AppServiceProvider::boot()` — lazy loading throws in local/testing (surfacing N+1s at dev time) but is silently allowed (logged, not thrown) in production so a missed case degrades performance rather than crashing a payroll run. Bulk operations (payslip generation across a run's entries, compliance-report row iteration) use `LazyCollection` (`Model::query()->lazy()`) instead of `->get()`, keeping memory flat regardless of employee count — the same reasoning the original applied to avoid loading an entire tenant's dataset into a single Node process's heap.

5.5 Partitioning, Archival, and Retention

`payroll_entries` and `audit_logs` are the two tables that grow unboundedly with tenant activity. Both are **range-partitioned by month** (`created_at`) using native Postgres declarative partitioning once a tenant's volume justifies it (Phase 2 signal, §37) — this keeps indexes small and lets old partitions be detached and archived to cold object storage instead of deleted, since payroll records typically carry multi-year statutory retention requirements (7 years is common across the three supported countries) that forbid deletion but not offlining.

```
CREATE TABLE audit_logs (
  -- columns...
) PARTITION BY RANGE (created_at);

CREATE TABLE audit_logs_2026_01 PARTITION OF audit_logs
FOR VALUES FROM ('2026-01-01') TO ('2026-02-01');
```

A scheduled Artisan command (`audit:partition-maintain`, Part 15) creates next month's partition ahead of time and detaches partitions older than the configured retention window, moving them to a `payrollfiti-archive` S3 bucket as compressed `COPY` dumps rather than dropping the data.

5.6 Backups Summary (full detail in Part 36)

Continuous WAL archiving to object storage gives point-in-time recovery; a base backup runs nightly. Given payroll data's legal-record status, RPO target is **under 5 minutes** (WAL shipping interval) and RTO target is **under 1 hour** for a full restore — both validated by a scheduled restore-drill, not assumed.

Part 6 — Laravel Application Architecture

6.1 Why Not Controllers → Services → Models

A flat `app/Http/Controllers` + `app/Services` + `app/Models` layout is Laravel's default tutorial shape, and it is exactly what NestJS's module system structurally prevented in the original: nothing stops `PayrollService` from directly manipulating a `Billing` model, and nothing signals to a new engineer where one domain's responsibility ends and another's begins. This guide instead uses explicit bounded-context folders, each independently testable and — per §2.3's Phase 3 — independently *extractable* without a rewrite.

6.2 Directory Structure

```
app/
├── Domain/
│   ├── Tenancy/           # Tenant, Company, tenant-resolution, RLS session variable binding
│   ├── Identity/         # User, roles/permissions, invitations, 2FA, sessions
│   ├── Employees/       # Employee, Contract, OnboardingTask
│   ├── Payroll/
│   │   ├── Engine/      # PURE – zero Eloquent/HTTP/queue imports (Part 9)
│   │   ├── Models/      # PayrollRun, PayrollEntry (Eloquent)
│   │   └── Rules/       # CountryRuleSet implementations (Part 9 §9.5)
│   ├── Compliance/     # Country-specific statutory report generation (Part 11)
│   ├── Leave/
│   ├── Loans/
│   ├── Attendance/
│   ├── Documents/
│   ├── Billing/         # Plan, Subscription, Invoice, UsageRecord
│   ├── Payments/       # PaymentTransaction, PaymentProvider contract + Paystack/M-Pesa
│   ├── Notifications/  # Notification, provider abstraction (email/SMS/push)
│   ├── Accounting/     # OAuth-integrated Xero/QuickBooks/Zoho sync
│   ├── Webhooks/       # ApiKey, WebhookEndpoint, WebhookDeliveryLog
│   ├── Audit/          # AuditLog + the Auditable trait/listener
│   └── Reporting/      # cross-domain read models for dashboard aggregate endpoints
├── Application/
│   ├── Commands/       # one class per state-changing use case, e.g. RunPayrollCommand
│   ├── Queries/        # one class per read use case that doesn't map 1:1 to a Resource
│   └── DTOs/           # framework-agnostic data carriers passed into/out of the Domain layer
├── Infrastructure/
│   ├── Persistence/    # Eloquent-specific repository implementations of Domain contracts
│   ├── Payments/       # Paystack/Daraja HTTP client wrappers
│   ├── Messaging/      # Mail/SMS/push provider client wrappers
│   ├── Storage/        # S3/local filesystem adapter configuration
│   ├── Monitoring/     # Prometheus exporter, Sentry context enrichment
│   └── ExternalServices/ # Accounting-platform OAuth clients
└── Http/
    ├── Controllers/Api/V1/ # thin – resolve request → dispatch Command/Query → return Resource
    ├── Middleware/
    ├── Requests/         # one Form Request per endpoint, owns validation + authorization gate
    └── Resources/        # one API Resource per response shape
```

Each `Domain/<Context>` is a self-registered Laravel package, not just a folder — it has its own `ServiceProvider` (`Domain\Payroll\PayrollServiceProvider`) registered in `bootstrap/providers.php`, which binds that domain's repository contracts to their `Infrastructure/Persistence` implementations and registers its own routes, migrations, and event listeners. This is what “each independently evolvable, independently extractable” means concretely: a domain's public surface is its Application-layer `Commands/Queries` and its published contracts — everything else (its Eloquent models, its internal services) is *internal* by PHP namespace convention and never imported directly from another `Domain/*` namespace.

6.3 Enforcing the Boundary in CI

Convention alone erodes under deadline pressure, so a static rule enforces it: `deptrac/deptrac` (a PHP architecture-linter) runs in CI (Part 34) with a ruleset stating each `Domain\X` layer may depend only on `Domain\X\*`, shared kernel types in `App\Shared`, and other domains' `Application\Commands/Queries/DTOs` — never another domain's `Models` or `Infrastructure` directly. A pull request that adds `use Domain\Billing\Models\Invoice;` inside `Domain\Payroll` fails CI with a clear violation message, the same hard stop NestJS's module system gave for free by refusing to resolve an unimported provider.

```
# deptrac.yaml (excerpt)
deptrac:
  layers:
    - {name: PayrollDomain, collectors: [{type: classLike, value: 'Domain\Payroll\*.'}]}
    - {name: BillingDomain, collectors: [{type: classLike, value: 'Domain\Billing\*.'}]}
    - {name: SharedApplication, collectors: [{type: classLike, value: 'Application\Commands|Queries|DTOs\*.'}]}
  ruleset:
    PayrollDomain: [SharedApplication] # Payroll may use shared Commands/Queries/DTOs, never Billing internals directly
    BillingDomain: [SharedApplication]
```

6.4 Cross-Domain Communication

Two domains coordinate through one of three sanctioned channels, chosen per §17 of the original request:

Mechanism	When to use	Example
Direct service call (via a published Application Command/Query, dependency-injected)	Synchronous, in-request, the caller needs the result immediately	<code>EmployeeController</code> calling <code>Domain\Payroll\Application\Queries\GetPayrollHistoryForEmployee</code>
Domain event → queued listener	The triggering domain doesn't need to know or wait for what happens next	<code>PayrollRunCompleted</code> event → <code>GeneratePayslips</code> , <code>DispatchWebhooks</code> , <code>SyncAccounting</code> listeners (Part 13 §13.5)
Scheduled command	Time-driven, not triggered by another domain's action	<code>leave:accrue-monthly</code> , <code>billing:process-subscription-renewals</code> (Part 15, Part 36)

Avoid: a fourth “just call the other domain's Eloquent model directly because it's convenient” path — this is exactly the erosion this structure exists to prevent, and it's what `deptrac` blocks mechanically rather than relying on code review discipline alone.

Part 7 — Authentication & Authorization

7.1 Authentication Modes

Two modes, both via Sanctum, matching §2.4's analysis:

Dashboard (Next.js, cookie-based SPA auth). The frontend calls `GET /sanctum/csrf-cookie` once, then `POST /login` with credentials; Sanctum issues an `httpOnly`, `Secure`, `SameSite=Lax` session cookie. Every subsequent request from the Next.js origin is authenticated by that cookie automatically, with Sanctum's `EnsureFrontendRequestsAreStateful` middleware validating the request's origin against `SANCTUM_STATEFUL_DOMAINS`.

Public API (Part 18 API keys). Authorization: `Bearer <token>` using Sanctum personal access tokens with scoped `abilities()` (`employees:read`, `payroll:run`, etc.) — no cookies, no CSRF concern, revocable individually and independently auditable via `last_used_at`.

```
// routes/api.php
Route::middleware('auth:sanctum')->prefix('v1')->group(function () {
    // reached by BOTH cookie-authenticated dashboard users AND bearer-token API clients —
    // Sanctum's guard resolves whichever credential is present
    Route::apiResource('employees', EmployeeController::class);
    Route::post('payroll/runs', [PayrollRunController::class, 'store'])
        ->middleware('ability:payroll:run'); // no-op for cookie sessions, enforced for API tokens
});
```

7.2 Login Flow, End to End

```
sequenceDiagram
    participant FE as Next.js
    participant API as Laravel API
    participant DB as PostgreSQL
    participant Redis as Redis (sessions)

    FE->>API: GET /sanctum/csrf-cookie
    API->>FE: Set-Cookie: XSRF-TOKEN
    FE->>API: POST /login {email, password}
    API->>DB: lookup user by email
    API->>API: Hash::check(password)
    alt 2FA enabled (Part 17)
        API->>FE: 200 {requires_2fa: true, challenge_id}
        FE->>API: POST /login/2fa {challenge_id, code}
        API->>API: verify TOTP / recovery code
    end
    API->>Redis: create session record
    API->>FE: Set-Cookie: laravel_session (httpOnly)
    FE->>API: GET /user (subsequent requests, cookie auto-sent)
    API->>FE: 200 {user, tenant, roles, permissions}
```

Rate limiting on `/login` uses Laravel's `RateLimiter` keyed by `email + ip`, not `ip` alone, matching the original's brute-force protection intent while avoiding a shared-office-IP false positive: 5 attempts per minute per key, then a 60-second lockout with exponential backoff on repeated violations (`throttle:login` custom limiter registered in `AppServiceProvider`).

7.3 Password Reset & Email Verification

Laravel's built-in `Illuminate\Auth\Notifications\ResetPassword` and `VerifyEmail` notifications are used as-is (queued, Part 13), with the `reset-token` table (`password_reset_tokens`) unchanged from Laravel's default — this is one of the areas where Laravel's out-of-the-box mechanism is already exactly what the original hand-built (“One Flow for Every Role” — admin, HR, employee all reset through the same endpoint, differentiated only by the `guard` context resolved from the token's associated user, not by separate flows).

7.4 Employee Invitations

```
HR action: POST /employees/{id}/invite
↓
InviteEmployeeCommand
↓
Employee.invitation_token (random 64-char, hashed at rest, 7-day expiry)
↓
Queued: SendEmployeeInvitationEmail (Part 13)
↓
Employee clicks link → GET /invitations/{token} (public, unauthenticated)
↓
Frontend renders "set your password" form
↓
POST /invitations/{token}/accept {password}
↓
AcceptInvitationCommand: hash password, clear token, mark User.email_verified_at, log in
```

The invitation token is hashed at rest (`hash('sha256', $rawToken)`, comparing the hash) — the same principle applied to password-reset tokens and API keys (Part 18) throughout this system: no secret token is ever stored in plaintext, so a database read (backup leak, replica misconfiguration) never directly yields a usable credential.

7.5 Session Management (Part 17 detail; overview here)

Sessions are stored in **Redis**, not the database, for two reasons: horizontal API scaling (any instance can validate any session, §37) and because session data doesn't carry the durability requirements that justify Postgres. Each session row exposed to the user (`GET /account/sessions`) shows device/browser (parsed from `User-Agent`), IP, and last-active time, with a “revoke” action per session and “revoke all others” — replacing the original's single-shared-token-slot model (Part 13/16 in the source document) with Sanctum's native multi-token/multi-session support, which handles the “revoke one device without logging out everywhere” requirement structurally rather than needing a custom refactor to retrofit it.

7.6 RBAC: Two Independent Checks, Composed

Original (NestJS)	Laravel equivalent	What it checks
RolesGuard	<code>Gate::before()</code> global check + <code>authorize()</code> calls against role	Coarse: is this user an <code>admin</code> , <code>hr</code> , or <code>employee</code> ?
PermissionsGuard	Model Policies	Fine-grained: can <i>this</i> user perform <i>this</i> action on <i>this specific resource instance</i> ?

```
// app/Domain/Payroll/Policies/PayrollRunPolicy.php
final class PayrollRunPolicy
{
    public function run(User $user, Company $company): bool
    {
        return $user->hasRole('admin')
            && $user->hasPermissionTo('payroll:run')
            && $user->tenant_id === $company->tenant_id; // belt-and-suspenders alongside the global scope, §8.4
    }

    public function approve(User $user, PayrollRun $run): bool
    {
        return $user->hasRole('admin')
            && $user->hasPermissionTo('payroll:approve')
            && $user->id !== $run->initiated_by; // segregation of duties: initiator cannot self-approve
    }

    public function view(User $user, PayrollRun $run): bool
    {
        return $user->tenant_id === $run->tenant_id
            && ($user->hasPermissionTo('payroll:read') || $user->id === $run->employeeUserId());
    }
}
```

```
// app/Http/Requests/Api/V1/RunPayrollRequest.php
final class RunPayrollRequest extends FormRequest
{
    public function authorize(): bool
    {
        return Gate::allows('run', $this->route('company'));
    }

    public function rules(): array
    {
        return [
            'period_start' => ['required', 'date'],
            'period_end'   => ['required', 'date', 'after_or_equal:period_start'],
            'pay_date'     => ['required', 'date', 'after_or_equal:period_end'],
        ];
    }
}
```

Both checks are declarative and co-located with the resource they protect (a Policy method, not a controller `if`), which is how this design prevents authorization logic from leaking into controllers or — worse — the frontend: **the frontend never decides who can do what**; it only reflects the `permissions` array returned on `/user` to conditionally render UI, and every mutating request is re-checked server-side regardless of what the UI showed.

7.7 Field-Level Encryption

National ID numbers, bank account numbers, and other PII that doesn't need to be queried (only displayed/exported) use a Laravel model cast backed by `Crypt::encryptString()/decryptString()` (AES-256-CBC, application `APP_KEY`-derived), transparent at the Eloquent layer:

```
// app/Domain/Employees/Models/Employee.php
protected function casts(): array
{
    return [
        'national_id' => 'encrypted',
        'bank_account_number' => 'encrypted',
        'hire_date' => 'date',
    ];
}
```

This matches the original's field-level encryption requirement exactly — Eloquent's `encrypted` cast is the framework-native version of the same “encrypt in the application layer, not just at rest via disk encryption” mechanism, with the key difference that key rotation requires a re-encryption migration pass (documented in Part 38) since Laravel does not

natively support multiple active decryption keys the way some KMS-integrated approaches do; this is accepted for Phase 1–2 and revisited if the security posture requires envelope encryption with a managed KMS (AWS KMS/Secrets Manager, §38.4) at Phase 3 scale.

Part 8 — Multi-Tenancy

8.1 Isolation Model Selection

Option	Cross-tenant leak risk	Operational complexity	Cost at scale	Verdict
Database per tenant	Lowest (physically impossible to cross-query without explicit multi-DB code)	Highest — migrations, connection pooling, and backups all multiply per tenant	Expensive past a few hundred tenants; connection-pool exhaustion becomes the binding constraint	Rejected — PayrollFiti's tenant count and per-tenant size profile (SMEs, not enterprises needing dedicated infrastructure) don't justify this
Shared database, separate schema per tenant	Low	Medium — migrations run per-schema, some ORMs handle this awkwardly	Better than per-DB, still multiplies migration/connection overhead	Rejected — Eloquent's schema-switching support is weaker than Prisma's was for this pattern, and Postgres RLS gives comparable isolation guarantees at a fraction of the operational cost
Shared database, shared schema, <code>tenant_id</code> column + structural scoping	Requires structural enforcement (below) to be safe	Lowest — one schema, one migration path, one connection pool	Scales cleanly to thousands of tenants on one Postgres instance/cluster	Selected — matches the original's decision, and Postgres RLS closes the original's main residual risk (a forgotten <code>WHERE tenant_id</code>) at the database layer

8.2 Two-Layer Structural Enforcement

Layer 1 — Application: a global Eloquent scope, bound from request context.

```
// app/Domain/Tenancy/TenantScope.php
final class TenantScope implements Scope
{
    public function apply(Builder $builder, Model $model): void
    {
        if ($tenantId = app(TenantContext::class)->id()) {
            $builder->where($model->getTable() . '.tenant_id', $tenantId);
        }
    }
}

// app/Domain/Tenancy/BelongsToTenant.php — applied via trait on every tenant-scoped model
trait BelongsToTenant
{
    protected static function bootBelongsToTenant(): void
    {
        static::addGlobalScope(new TenantScope);
        static::creating(function (Model $model) {
            $model->tenant_id ??= app(TenantContext::class)->id();
        });
    }
}
```

`TenantContext` is a simple, request-scoped singleton — **not** a global/static — resolved from the authenticated user's `tenant_id` (dashboard sessions) or the API key's owning tenant (public API), bound early in the middleware pipeline:

```
// app/Http/Middleware/ResolveTenantContext.php
final class ResolveTenantContext
{
    public function handle(Request $request, Closure $next): Response
    {
        $tenantId = $request->user()?->tenant_id
            ?? $request->user()?->currentAccessToken()?->tenant_id;

        abort_unless($tenantId, 401, 'No tenant context resolved.');
```

```
        app(TenantContext::class)->set($tenantId);

        // Layer 2: bind the same value as a Postgres session variable for this request's
        // connection lifetime, consumed by RLS policies below.
        DB::statement('SET app.current_tenant_id = ?', [$tenantId]);

        return $next($request);
    }
}
```

Layer 2 — Database: PostgreSQL Row-Level Security, as defense-in-depth against anything that bypasses the Eloquent scope — a raw `DB::select()`, a forgotten `withoutGlobalScope()`, a third-party package's internal query:

```
ALTER TABLE employees ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON employees
    USING (tenant_id = current_setting('app.current_tenant_id', true)::uuid);

-- Applied via a migration helper run for every tenant-scoped table:
-- app/Domain/Tenancy/Migrations/EnablesRowLevelSecurity trait
```

```
// database/migrations/..._enable_rls_on_tenant_tables.php
foreach (TenantScopedTables::all() as $table) {
    DB::statement("ALTER TABLE {$table} ENABLE ROW LEVEL SECURITY");
    DB::statement("
        CREATE POLICY tenant_isolation ON {$table}
        USING (tenant_id = current_setting('app.current_tenant_id', true)::uuid)
    ");
}
```

This is the redesign's single biggest structural improvement over the original: the Prisma client extension was the *only* enforcement layer, so any code path that constructed a raw query or used a different Prisma client instance bypassed it silently. RLS makes that failure mode a database-level rejection (zero rows returned, or an insert violation) instead of a silent leak — it fails closed.

Architectural Trade-offs

Decision	Chosen Approach	Alternative	Why Chosen	Cost/Tradeoff
Tenant scoping	App scope + Postgres RLS (two layers)	App scope only	RLS closes the “forgot to scope” failure mode at the layer nothing can bypass	Every tenant-scoped connection must <code>SET app.current_tenant_id</code> before querying — a missed <code>SET</code> (e.g., a queue worker reusing a pooled connection between jobs for different tenants) fails closed (returns zero rows) rather than open, but this requires the connection-reset discipline in §8.5 to avoid confusing “no data” bugs
Tenant context storage		PHP superglobals / static properties	Testable (easy to swap in tests), no risk of one request's context leaking	Every entry point that isn't standard HTTP middleware (queue jobs,

Decision	Chosen Approach	Alternative	Why Chosen	Cost/Tradeoff
	Request-scoped singleton, explicitly set by middleware		into another <i>within the same PHP-FPM worker process</i> , unlike a static property which persists across requests handled by the same worker unless explicitly reset	Artisan commands, scheduled tasks) must explicitly resolve and set tenant context itself — handled in §8.5

8.3 Cross-Tenant Queries (the Legitimate Exceptions)

A small, explicit allowlist of operations legitimately need cross-tenant visibility — platform admin tooling, billing reconciliation across all tenants, and the public payroll calculator (Part 10 §10.5, which runs the pure engine with no tenant context at all since it never touches persisted data). These use an explicit `Model::withoutTenantScope()` helper that requires an accompanying `Gate::authorize('platform-admin')` check — never a bare `withoutGlobalScope()` call, which would be invisible in a code review diff. `deptrac` and a custom PHPStan rule (Part 6 §6.3, Part 24) flag any `withoutGlobalScope(TenantScope::class)` call that isn't routed through this helper.

8.4 Queues, Scheduled Jobs, and CLI Commands

Tenant context does **not** survive process boundaries automatically — a queued job runs in a different PHP process than the request that dispatched it. Every job that touches tenant-scoped data carries its `tenant_id` explicitly as a constructor property (never inferred from ambient state) and re-establishes both enforcement layers at the start of its `handle()` method:

```
// app/Domain/Payroll/Jobs/GeneratePayslip.php
final class GeneratePayslip implements ShouldQueue
{
    public function __construct(
        public readonly string $tenantId,
        public readonly string $payrollEntryId,
    ) {}

    public function handle(TenantContext $context): void
    {
        $context->set($this->tenantId);
        DB::statement("SET app.current_tenant_id = ?", [$this->tenantId]);

        // ... generate PDF, store, dispatch delivery job
    }

    public function middleware(): array
    {
        return [new WithoutOverlapping($this->payrollEntryId)]; // idempotency, Part 13 §13.4
    }
}
```

A base `TenantAwareJob` abstract class wraps this pattern so individual job classes don't repeat it — and a Horizon-registered `after` callback resets the Postgres session variable (`RESET app.current_tenant_id`) after every job completes, since Horizon workers reuse database connections across jobs from a pool; without this reset, a worker could carry Tenant A's session variable into the next job's connection if that job forgets to `SET` it explicitly, which is why the `TenantAwareJob` base class is mandatory (enforced by a PHPStan rule requiring every `ShouldQueue` class touching tenant-scoped models to extend it) rather than optional convention.

Scheduled commands that operate tenant-by-tenant (billing renewal checks, leave accrual, Part 15/36) iterate tenants explicitly and set context per-iteration; platform-wide maintenance commands (partition maintenance, Part 5 §5.5) are the intentional cross-tenant exception, gated the same way as §8.3.

8.5 Storage, Caching, and Notification Isolation

- **Object storage:** every S3 key is prefixed `tenants/{tenant_id}/...`; bucket policies additionally restrict signed-URL generation to keys under the requesting tenant's prefix, so even a bug generating a wrong key still can't produce a valid signed URL into another tenant's prefix.
 - **Cache:** every cache key used for tenant-scoped data is prefixed `tenant:{tenant_id}:...` by a `TenantAwareCache` wrapper around Laravel's `Cache` facade; a shared Redis cache without this prefix would otherwise be the one place tenant data could collide (two tenants both caching “employee count” under the same literal key).
 - **Webhooks/API keys/notifications:** `webhook_endpoints`, `api_keys`, and `notifications` all carry `tenant_id` and go through the same `BelongsToTenant` trait and RLS policy as every other table — there is no separate code path for these, which is deliberate: a special case is where isolation bugs hide.
-

Part 9 — Payroll Calculation Engine

9.1 The Non-Negotiable Constraint

Everything in this part exists to preserve one property, unchanged from the original: **the payroll engine is pure**. It performs no I/O, reads no clock, generates no randomness, and imports nothing from Eloquent, HTTP, or the queue system. Given the same `PayrollInput` and the same `CountryRuleSet` version, it returns byte-identical output — forever. This is what makes historical payroll runs legally reproducible and what lets the same code power both the authenticated payroll run and the public, unauthenticated marketing-site calculator (\$10.5).

In PHP, “pure” additionally means: no static state, no `now()` / `Carbon::now()` calls inside the engine (dates always arrive as explicit constructor arguments), and — critically — **no Eloquent imports anywhere under `Domain/Payroll/Engine`**. This is enforced by the same `deptrac` ruleset from Part 6 §6.3, with a dedicated layer:

```
# deptrac.yaml addition
layers:
  - {name: PayrollEngine, collectors: [{type: classLike, value: 'Domain\\Payroll\\Engine\\.*'}]}
ruleset:
  PayrollEngine: [] # depends on NOTHING outside itself — not even other Domain layers
```

9.2 Namespace Layout

```
app/Domain/Payroll/Engine/
├── PayrollInput.php           # readonly DTO – every field the engine needs
├── PayrollResult.php          # readonly DTO – every field the engine produces
├── Money.php                  # round2/round4/sum – bcmath-backed, not float
├── Proration.php              # mid-period joiner/leaver math
├── PayrollCalculator.php       # calculate() – the single entrypoint
├── RuleRegistry.php           # country + version resolution
├── Contracts/
│   └── CountryRuleSet.php      # interface every country implements
├── Rules/
│   ├── Kenya/               # Kenya2024RuleSet.php, Kenya2025RuleSet.php, PAYE.php, NSSF.php,
│   │   ├── SHIF.php, HousingLevy.php
│   │   ├── Nigeria/          # Nigeria2024RuleSet.php, PAYE.php, Pension.php, NHF.php
│   │   └── SouthAfrica/       # SouthAfrica2024RuleSet.php, PAYE.php, UIF.php, SDL.php
```

9.3 Core Types

PHP 8.3 readonly classes and native enums replace the original’s TypeScript interfaces — the contract is identical in shape, just expressed with PHP’s value-object idioms instead of structural typing:

```
// app/Domain/Payroll/Engine/PayrollInput.php
final readonly class PayrollInput
{
    /**
     * @param array<string,float> $allowances
     * @param array<string,float> $voluntaryDeductions
     */
    public function __construct(
        public string $employeeId,
        public string $countryCode,
        public string $currency,
        public float $basicSalary,
        public array $allowances,
        public float $overtimeAmount,
        public float $commissionAmount,
        public float $bonusAmount,
        public array $voluntaryDeductions,
        public DateTimeImmutable $periodStart,
        public DateTimeImmutable $periodEnd,
        public ?DateTimeImmutable $employmentStartDate = null,
        public ?DateTimeImmutable $employmentEndDate = null,
    ) {}
}

// app/Domain/Payroll/Engine/Contracts/CountryRuleSet.php
interface CountryRuleSet
{
    public function countryCode(): string;
    public function version(): string; // opaque tag recorded on every payroll run, e.g. "KE-2025"
    public function effectiveFrom(): DateTimeImmutable;
    public function currency(): string;

    public function calculateEarnings(PayrollInput $input): EarningsResult;

    /** @return StatutoryDeductionLine[] */
    public function calculateStatutoryDeductions(float $grossPay, EarningsResult $earnings): array;

    public function calculateTax(float $grossPay, array $statutoryDeductions): TaxResult;

    /** @return ValidationIssue[] */
    public function validate(PayrollInput $input): array;
}

// app/Domain/Payroll/Engine/StatutoryDeductionLine.php
final readonly class StatutoryDeductionLine
{
    public function __construct(
        public string $code, // "NSSF", "PENSION", "UIF", ...
        public string $label,
        public float $employeeAmount,
        public float $employerAmount,
    ) {}
}
```

9.4 Money & Determinism Helpers

Floating-point money math is a correctness bug waiting to happen in any language; PHP's `bcmath` extension (arbitrary-precision decimal arithmetic) replaces the original's careful `Math.round` discipline with arithmetic that cannot represent `0.1 + 0.2` incorrectly in the first place — a genuine improvement Laravel/PHP makes easy to adopt, not just a translation:

```
// app/Domain/Payroll/Engine/Money.php
final class Money
{
    public static function round2(float $value): float
    {
        return (float) bcadd((string) $value, '0', 2); // 2 decimal places, half-up
    }

    public static function sum(array $values): float
    {
        return self::round2(array_reduce($values, fn ($carry, $v) => $carry + $v, 0.0));
    }

    /**
     * Deterministic, key-order-independent JSON serialization — sorts keys
     * recursively so the same logical input always hashes identically
     * regardless of array-construction order.
     */
    public static function stableJson(mixed $value): string
    {
        if (is_array($value)) {
            $isList = array_is_list($value);
            if (! $isList) {
                ksort($value);
            }
            $items = array_map(
                fn ($k, $v) => $isList ? self::stableJson($v) : json_encode((string) $k) . ':' . self::stableJson($v),
                array_keys($value),
                $value,
            );
            return $isList ? '[' . implode(',', $items) . ']' : '{' . implode(',', $items) . '}';
        }
        return json_encode($value);
    }
}
```

Every intermediate monetary value is rounded via `Money::round2()` immediately after computing it — never accumulated unrounded and rounded once at the end — so a payslip's line items always sum exactly to its displayed totals, exactly as the original required.

9.5 Proration

```
// app/Domain/Payroll/Engine/Proration.php
final class Proration
{
    public static function factor(PayrollInput $input): float
    {
        $totalDays = self::diffDaysInclusive($input->periodStart, $input->periodEnd);
        if ($totalDays <= 0) {
            return 0.0;
        }

        $effectiveStart = $input->employmentStartDate && $input->employmentStartDate > $input->periodStart
            ? $input->employmentStartDate : $input->periodStart;
        $effectiveEnd = $input->employmentEndDate && $input->employmentEndDate < $input->periodEnd
            ? $input->employmentEndDate : $input->periodEnd;

        if ($effectiveEnd < $effectiveStart) {
            return 0.0;
        }

        $workedDays = self::diffDaysInclusive($effectiveStart, $effectiveEnd);
        return min(1.0, max(0.0, round($workedDays / $totalDays, 4)));
    }

    private static function diffDaysInclusive(DateTimeImmutable $start, DateTimeImmutable $end): int
    {
        return (int) $start->diff($end)->days + 1;
    }
}
```

Overtime, commission, and bonus amounts remain deliberately **unscaled** by this factor — carried over verbatim from the original's reasoning: they represent actuals already tied to real days worked or one-off events, not a rate that should be diluted by partial attendance. Only `basicSalary` and `allowances` are prorated.

9.6 The Engine Entrypoint

```
// app/Domain/Payroll/Engine/PayrollCalculator.php
final class PayrollCalculator
{
    public function calculate(PayrollInput $input, CountryRuleSet $ruleSet): PayrollResult
    {
        $validation = $ruleSet->validate($input);
        $prorationFactor = Proration::factor($input);

        $fullEarnings = $ruleSet->calculateEarnings($input);
        $earnings = $fullEarnings->prorate($prorationFactor); // scales basic + allowances only

        $statutoryDeductions = $ruleSet->calculateStatutoryDeductions($earnings->grossPay, $earnings);
        $tax = $ruleSet->calculateTax($earnings->grossPay, $statutoryDeductions);

        $totalVoluntary = Money::sum(array_values($input->voluntaryDeductions));
        $totalStatutoryEmployee = Money::sum(array_map(fn ($d) => $d->employeeAmount, $statutoryDeductions));
        $totalDeductions = Money::round2($totalStatutoryEmployee + $tax->netTax + $totalVoluntary);
        $netPay = Money::round2($earnings->grossPay - $totalDeductions);

        return new PayrollResult(
            employeeId: $input->employeeId,
            countryCode: $ruleSet->countryCode(),
            currency: $input->currency,
            ruleVersion: $ruleSet->version(),
            prorationFactor: $prorationFactor,
            earnings: $earnings,
            statutoryDeductions: $statutoryDeductions,
            voluntaryDeductions: $input->voluntaryDeductions,
            totalVoluntaryDeductions: $totalVoluntary,
            tax: $tax,
            totalDeductions: $totalDeductions,
            grossPay: $earnings->grossPay,
            netPay: $netPay,
            validation: $validation,
            inputHash: $this->computeInputHash($input, $ruleSet->version()),
        );
    }

    public function computeInputHash(PayrollInput $input, string $ruleVersion): string
    {
        return hash('sha256', Money::stableJson(['input' => $input, 'ruleVersion' => $ruleVersion]));
    }
}
```

The order of operations is preserved exactly from the original and is load-bearing: proration happens *between* earnings calculation and everything downstream, so a mid-month joiner's statutory deductions and tax are always computed on their **prorated** gross pay — what they actually earned that period — never their full monthly rate.

9.7 The Registry — Versioned Rules by Country

```
// app/Domain/Payroll/Engine/RuleRegistry.php
final class RuleRegistry
{
    /** @var array<string, CountryRuleSet[]> */
    private array $ruleSets;

    public function __construct()
    {
        $this->ruleSets = [
            'KE' => [new Rules\Kenya\Kenya2024RuleSet, new Rules\Kenya\Kenya2025RuleSet],
            'NG' => [new Rules\Nigeria\Nigeria2024RuleSet],
            'ZA' => [new Rules\SouthAfrica\SouthAfrica2024RuleSet],
        ];
    }

    /**
     * Resolves the rule set applicable for a country on a given date, picking
     * the most recent version whose effectiveFrom is on or before that date.
     * A payroll run always records which version it used (payroll_runs.rule_version),
     * and re-running the same period later reproduces it exactly by passing the
     * PERIOD date as $effectiveDate — never `now()`.
     */
    public function resolve(string $countryCode, DateTimeImmutable $effectiveDate): CountryRuleSet
    {
        $candidates = $this->ruleSets[strtoupper($countryCode)] ?? throw new UnsupportedCountryException($countryCode);

        $applicable = array_filter($candidates, fn (CountryRuleSet $rs) => $rs->effectiveFrom() <= $effectiveDate);
        usort($applicable, fn ($a, $b) => $b->effectiveFrom() <=> $a->effectiveFrom());

        return $applicable[0] ?? throw new NoApplicableRuleVersionException($countryCode, $effectiveDate);
    }

    public function byVersion(string $countryCode, string $version): CountryRuleSet
    {
        foreach ($this->ruleSets[strtoupper($countryCode)] ?? [] as $rs) {
            if ($rs->version() === $version) {
                return $rs;
            }
        }
        throw new UnsupportedCountryException("{ $countryCode }@{ $version }");
    }
}
```

Adding a fourth country is exactly: write a new `Rules/<Country>/` namespace implementing `CountryRuleSet`, register it in `RuleRegistry`'s constructor array. Nothing else in the engine, the API, the queue jobs, or the frontend calculator needs to change — the same closed-for-modification, open-for-extension property the original's strategy pattern gave.

9.8 Kenya (KE) — Preserving the NHIF → SHIF Transition

Kenya remains the most complex ruleset because it carries a real legislative transition: NHIF was replaced by SHIF (Social Health Insurance Fund) on 1 October 2024, and a system that reproduces historical payslips must keep both versions live forever, unmodified.

```
// app/Domain/Payroll/Engine/Rules/Kenya/Kenya2024RuleSet.php
final class Kenya2024RuleSet implements CountryRuleSet
{
    public function countryCode(): string { return 'KE'; }
    public function version(): string { return 'KE-2024'; }
    public function effectiveFrom(): DateTimeImmutable { return new DateTimeImmutable('2024-01-01'); }
    public function currency(): string { return 'KES'; }

    public function calculateStatutoryDeductions(float $grossPay, EarningsResult $earnings): array
    {
        return [
            NSSF::calculate($grossPay),
            NHIF::calculate($grossPay), // legacy scheme — frozen, never touched again
            HousingLevy::calculate($grossPay),
        ];
    }
    // calculateEarnings(), calculateTax(), validate() — same shape, omitted for brevity per §61
}

// app/Domain/Payroll/Engine/Rules/Kenya/Kenya2025RuleSet.php
final class Kenya2025RuleSet implements CountryRuleSet
{
    public function version(): string { return 'KE-2025'; }
    public function effectiveFrom(): DateTimeImmutable { return new DateTimeImmutable('2024-10-01'); } // SHIF cutover

    public function calculateStatutoryDeductions(float $grossPay, EarningsResult $earnings): array
    {
        return [
            NSSF::calculate($grossPay),
            SHIF::calculate($grossPay), // replaces NHIF from this version forward
            HousingLevy::calculate($grossPay),
        ];
    }
}
```

`Kenya2024RuleSet` is never edited to “fix” it after `Kenya2025RuleSet` ships — this is the same immutable-versioning discipline the original enforced, because editing it would silently change the recomputed result of every historical March-2024 payslip.

9.9 Idempotency: `input_hash` as a Structural Guarantee

`PayrollCalculator::computeInputHash()` feeds directly into the `payroll_runs.input_hash` unique constraint from Part 5 §5.2 (`UNIQUE(tenant_id, company_id, input_hash)`). Submitting the same payroll run twice — a network retry, a double-click, a queue redelivery — produces the same hash, and the database’s unique constraint rejects the duplicate insert at the storage layer, not just an application-level check-then-insert that has a race window. The `RunPayrollCommand` (Part 10 §10.2) catches the resulting unique-constraint violation and returns the **existing** run rather than erroring, which is what makes payroll execution genuinely idempotent under concurrent duplicate requests — Part 32 §32.7’s race-condition test verifies this explicitly.

9.10 Testing the Engine

Because the engine has zero framework dependencies, its test suite runs with Pest’s plain unit-testing mode — no database, no HTTP kernel, sub-millisecond per test:

```
// tests/Unit/Domain/Payroll/Engine/Kenya2025RuleSetTest.php
it('computes SHIF instead of NHIF for periods on or after 1 Oct 2024', function () {
    $ruleSet = new Kenya2025RuleSet;
    $input = PayrollInputFactory::make(countryCode: 'KE', basicSalary: 80_000.00);

    $result = app(PayrollCalculator::class)->calculate($input, $ruleSet);

    expect($result->statutoryDeductions)->toContainCode('SHIF')
        ->and($result->statutoryDeductions)->not->toContainCode('NHIF')
        ->and($result->netPay)->toBe(round($result->grossPay - $result->totalDeductions, 2));
});

it('produces byte-identical output for the same input twice', function () {
    $input = PayrollInputFactory::make();
    $ruleSet = new Kenya2025RuleSet;
    $calculator = app(PayrollCalculator::class);

    expect($calculator->calculate($input, $ruleSet)->inputHash)
        ->toBe($calculator->calculate($input, $ruleSet)->inputHash);
});

it('resolves KE-2024 for a period before the SHIF cutover and KE-2025 after', function () {
    $registry = new RuleRegistry;

    expect($registry->resolve('KE', new DateTimeImmutable('2024-09-15'))->version())->toBe('KE-2024')
        ->and($registry->resolve('KE', new DateTimeImmutable('2024-10-15'))->version())->toBe('KE-2025');
});
```

Part 10 — Payroll API

10.1 Execution Flow: Request → Command → Engine → Persistence → Events

```
sequenceDiagram
    participant FE as Next.js
    participant C as PayrollRunController
    participant CMD as RunPayrollCommand
    participant ENG as PayrollCalculator (pure)
    participant DB as PostgreSQL
    participant Q as Queue

    FE->>C: POST /v1/companies/{c}/payroll/runs
    C->>C: RunPayrollRequest: validate + Gate::authorize('run', $company)
    C->>CMD: handle(company, period, initiatedBy)
    loop for each active employee
        CMD->>ENG: calculate(PayrollInput, resolvedRuleSet)
    end
    CMD->>DB: BEGIN
    CMD->>DB: INSERT payroll_runs (status=processing, input_hash, rule_version)
    alt input_hash already exists (duplicate submission)
        DB-->>CMD: unique constraint violation
        CMD-->>C: return EXISTING run (idempotent no-op)
    else
        CMD->>DB: INSERT payroll_entries (one per employee, immutable)
        CMD->>DB: UPDATE payroll_runs SET status=completed
        CMD->>DB: COMMIT
        CMD->>Q: dispatch PayrollRunCompleted event
    end
    C-->>FE: 201 {payroll_run}
```

10.2 The Command

```
// app/Domain/Payroll/Application/Commands/RunPayrollCommand.php
final class RunPayrollCommand
{
    public function __construct(
        private readonly RuleRegistry $registry,
        private readonly PayrollCalculator $calculator,
        private readonly EmployeeRepository $employees,
    ) {}

    public function handle(Company $company, CarbonImmutable $periodStart, CarbonImmutable $periodEnd, User $initiator): PayrollRun
    {
        $activeEmployees = $this->employees->activeForCompany($company);
        $ruleSet = $this->registry->resolve($company->country, $periodStart->toDateTimeImmutable());

        $results = $activeEmployees->map(fn (Employee $employee) => $this->calculator->calculate(
            $this->buildInput($employee, $periodStart, $periodEnd),
            $ruleSet,
        ));

        $runInputHash = hash('sha256', Money::stableJson($results->pluck('inputHash')->sort()->values()->all()));

        return DB::transaction(function () use ($company, $periodStart, $periodEnd, $ruleSet, $results, $runInputHash, $initiator) {
            try {
                $run = PayrollRun::create([
                    'company_id' => $company->id,
                    'period_start' => $periodStart,
                    'period_end' => $periodEnd,
                    'input_hash' => $runInputHash,
                    'rule_version' => $ruleSet->version(),
                    'status' => 'completed',
                    'initiated_by' => $initiator->id,
                ]);
            } catch (UniqueConstraintViolationException) {
                // Idempotent replay: return the run that already exists for this exact input.
                return PayrollRun::where('company_id', $company->id)->where('input_hash', $runInputHash)->firstOrFail();
            }

            $run->entries()->insert($results->map(fn ($r, $i) => [
                'id' => Str::uuid(),
                'tenant_id' => $company->tenant_id,
                'employee_id' => $activeEmployees[$i]->id,
                'gross_pay' => $r->grossPay,
                'net_pay' => $r->netPay,
                'total_deductions' => $r->totalDeductions,
                'breakdown' => json_encode($r), // full immutable snapshot — Part 5 §5.2
                'created_at' => now(),
            ]->all()));

            event(new PayrollRunCompleted($run));

            return $run;
        });
    }
}
```

10.3 Payroll Corrections

Corrections never mutate a historical `payroll_entries` row (Part 5 §5.2 deliberately omits `updated_at` from that table). A correction is a **new** `PayrollRun` with `corrects_run_id` set, going through the identical command above — the correction's `input_hash` differs from the original (different inputs by definition), so it is never rejected as a duplicate, and both runs remain permanently queryable, giving a full audit trail of what changed and why (a required `reason` field on the correction endpoint feeds `AuditLog`, Part 6).

10.4 Concurrency: Two Simultaneous Executions of the Same Run

The original's race-condition requirement — “two requests attempt to execute the same payroll run simultaneously; only one execution succeeds” — is satisfied by two overlapping mechanisms, deliberately redundant:

1. **Application-level lock:** `Cache::lock("payroll-run:{$company->id}:{$periodStart}", 30)->block(5)` wraps the command, so a second concurrent request waits briefly and then observes the first request's committed result instead of racing it.
2. **Database-level guarantee:** even if the lock were somehow bypassed, the `UNIQUE(tenant_id, company_id, input_hash)` constraint from Part 5 makes a true double-insert impossible — the second transaction's `INSERT` fails and the `UniqueConstraintViolationException` branch in §10.2 returns the existing run.

The lock is an optimization (avoids redundant computation and a rolled-back transaction under contention); the unique constraint is the actual correctness guarantee. Part 32 §32.7 documents the test that exercises this with genuinely concurrent requests, not a simulated one.

10.5 Public Payroll Calculator

The marketing site's unauthenticated calculator (original §10.5) calls the **same** `PayrollCalculator::calculate()` used internally — there is no separate “demo” implementation to keep in sync. It is exposed as a public route with no tenant context, no persistence, and aggressive rate limiting (`throttle:public-calculator, 10/minute per IP`):

```
// routes/api.php
Route::post('v1/public/payroll-calculator', PublicPayrollCalculatorController::class)
    ->middleware('throttle:public-calculator');
```

```
final class PublicPayrollCalculatorController
{
    public function __invoke(PublicCalculatorRequest $request, RuleRegistry $registry, PayrollCalculator $calc): JsonResponse
    {
        $ruleSet = $registry->resolve($request->countryCode(), CarbonImmutable::now()->toDateTimeImmutable());
        $result = $calc->calculate($request->toPayrollInput(), $ruleSet);

        return response()->json(new PublicCalculatorResource($result)); // net pay + high-level breakdown only, no PII
    }
}
```

10.6 Representative Endpoint Documentation

POST `/api/v1/companies/{company}/payroll/runs`

Aspect	Detail
Auth	Sanctum (session or <code>payroll:run</code> token ability)
Permissions	<code>Gate::authorize('run', \$company)</code> — admin role + <code>payroll:run</code> permission (Part 7 §7.6)
Request	<code>{period_start, period_end, pay_date}</code>
Validation	<code>RunPayrollRequest</code> — dates ordered, period within company's active range
Response 201	<code>PayrollRunResource</code> — id, status, totals, <code>entries_count</code>
Response 200	Same shape, returned instead of 201 when the request is an idempotent replay (§10.4)
Errors	422 validation, 403 authorization, 409 if a prior run for an overlapping period is still <code>processing</code>
Tenant behavior	<code>company</code> route-model-bound and scoped by <code>TenantScope</code> (Part 8) — a company ID belonging to another tenant 404s, not 403, to avoid confirming its existence
Idempotency	Structural, via <code>input_hash</code> (§9.9, §10.4)

Aspect	Detail
DB interaction	One transaction: insert <code>payroll_runs</code> + bulk-insert <code>payroll_entries</code>
Events/jobs	<code>PayrollRunCompleted</code> → <code>GeneratePayslips</code> , <code>DispatchWebhooks</code> , <code>SyncAccounting</code> listeners (Part 13 §13.5)

Every other payroll endpoint (GET `.../payroll/runs`, GET `.../payroll/runs/{run}`, POST `.../payroll/runs/{run}/approve`, POST `.../payroll/runs/{run}/corrections`) follows this same documentation shape; full endpoint-by-endpoint tables for every domain are generated automatically from route + Form Request + Policy + Resource introspection into the OpenAPI spec (§43) rather than hand-maintained prose, which is also how the frontend's TypeScript types stay in sync without manual duplication.

Part 11 — Compliance & Country Rules

11.1 Design: Country-Gated, Template-Driven

Compliance reports (Kenya P9/P10/NSSF/SHIF remittance; Nigeria's equivalents; South Africa's EMP201/IRP5) are derived entirely from `PayrollRun / PayrollEntry` data already produced by Part 9–10 — no separate calculation path, matching the original's "same engine output, different presentation" design. Each report is a `Domain\Compliance\Reports\<Country>\<ReportCode>Report` class that reads a date range of `payroll_entries.breakdown` JSONB snapshots, aggregates per statutory line, and renders either a structured JSON export or a Blade view piped through `dompdf` (simple, tabular reports — §2.5's tool-matching rationale).

```
// app/Domain/Compliance/Reports/Kenya/P9Report.php
final class P9Report
{
    public function generate(Company $company, int $taxYear): P9ReportData
    {
        $entries = PayrollEntry::query()
            ->whereHas('payrollRun', fn ($q) => $q->where('company_id', $company->id)
                ->whereYear('period_start', $taxYear))
            ->get();

        return P9ReportData::fromEntries($entries); // pure aggregation over already-computed breakdowns
    }
}
```

11.2 Adding a Fourth Country's Compliance Reports

Unchanged in shape from the original: implement the country's report classes under `Domain\Compliance\Reports\<Country>`, register them in a `ComplianceReportRegistry` keyed by country code. Nothing in the payroll engine or the reporting queue jobs needs to change — reports are a read-side projection over data the engine already produced, never a second source of truth for statutory numbers.

Architectural Trade-offs

Decision	Chosen Approach	Alternative	Why Chosen	Cost/Tradeoff
Report generation trigger	On-demand (queued job per request)	Pre-generated on every payroll run completion	Compliance reports are requested far less often than payroll runs execute; generating on-demand avoids wasted work for reports nobody downloads	First download of a given period has a few seconds' latency (queued job + polling/webhook for completion) instead of instant — acceptable given reports are typically pulled at month/quarter end, not immediately after each run

Part 12 — Billing & Payments

12.1 Schema Recap & Provider Abstraction

`Plan`, `Subscription`, `Invoice`, `UsageRecord`, and `PaymentTransaction` carry over unchanged as Eloquent models under `Domain\Billing` and `Domain\Payments` respectively. The `PaymentProvider` interface replaces the original's TypeScript equivalent one-for-one:

```
// app/Domain/Payments/Contracts/PaymentProvider.php
interface PaymentProvider
{
    public function initiate(PaymentRequest $request): PaymentInitiationResult;
    public function verifyWebhookSignature(Request $request): bool;
    public function parseWebhook(Request $request): WebhookEvent;
    public function reconcile(string $providerReference): PaymentStatus; // active polling fallback, §12.5
}
```

```
// app/Infrastructure/Payments/PaystackProvider.php
final class PaystackProvider implements PaymentProvider
{
    public function __construct(private readonly PendingRequest $http) {}

    public function initiate(PaymentRequest $request): PaymentInitiationResult
    {
        $response = $this->http->post('https://api.paystack.co/transaction/initialize', [
            'email' => $request->payerEmail,
            'amount' => (int) round($request->amount * 100), // Paystack expects kobo/cents
            'reference' => $request->idempotencyKey, // same idempotency-key discipline as payroll runs
            'callback_url' => $request->callbackUrl,
        ]->throw());

        return PaymentInitiationResult::fromPaystack($response->json());
    }

    public function verifyWebhookSignature(Request $request): bool
    {
        $expected = hash_hmac('sha512', $request->getContent(), config('services.paystack.secret'));
        return hash_equals($expected, $request->header('x-paystack-signature', ''));
    }
}
```

```
// app/Infrastructure/Payments/MpesaProvider.php — STK Push
final class MpesaProvider implements PaymentProvider
{
    public function initiate(PaymentRequest $request): PaymentInitiationResult
    {
        $token = $this->getOAuthToken(); // Daraja OAuth, cached with TTL just under actual token expiry

        $response = $this->http->withToken($token)->post($this->stkPushUrl(), [
            'BusinessShortCode' => config('services.mpesa.shortcode'),
            'Password' => $this->generatePassword(),
            'Timestamp' => now()->format('YmdHis'),
            'TransactionType' => 'CustomerPayBillOnline',
            'Amount' => (int) round($request->amount),
            'PartyA' => $request->payerPhone,
            'PartyB' => config('services.mpesa.shortcode'),
            'PhoneNumber' => $request->payerPhone,
            'CallBackURL' => config('services.mpesa.callback_url'),
            'AccountReference' => $request->idempotencyKey,
        ]->throw();

        return PaymentInitiationResult::fromMpesa($response->json());
    }

    public function verifyWebhookSignature(Request $request): bool
    {
        // Daraja callbacks carry no HMAC signature by default — verified instead by a
        // shared bearer token in the callback URL path/query, configured at registration time.
        return hash_equals(config('services.mpesa.callback_token'), $request->query('token', ''));
    }
}
```

12.2 Never Trust a Client Redirect

The original's cardinal rule carries over unchanged: **a client-side redirect back from Paystack/M-Pesa is never treated as proof of payment.** `PaymentTransaction.status` only transitions to `succeeded` from one of two server-verified sources: (1) the provider's signed webhook, or (2) an explicit server-to-server reconciliation poll (§12.5). The frontend redirect only triggers the UI to *start polling* `GET /payments/{id}` for status — it never writes state.

12.3 Webhook Processing

```
// app/Http/Controllers/Api/V1/Webhooks/PaystackWebhookController.php
final class PaystackWebhookController
{
    public function __invoke(Request $request, PaystackProvider $provider): Response
    {
        abort_unless($provider->verifyWebhookSignature($request), 401);

        $event = $provider->parseWebhook($request);

        // Replay protection: the provider's event ID is the idempotency key, unique-constrained.
        $created = WebhookInboundEvent::firstOrCreate(
            ['provider' => 'paystack', 'provider_event_id' => $event->id],
            ['payload' => $request->all()],
        );
        if (! $created->wasRecentlyCreated) {
            return response()->noContent(); // duplicate delivery — already processed, ack and stop
        }

        ProcessPaymentWebhook::dispatch($event)->onQueue('webhooks'); // async, §13.3
        return response()->noContent();
    }
}
```

Duplicate callbacks (a documented Paystack/Daraja behavior under retry) are absorbed by the `provider_event_id` uniqueness check *before* the job is even queued, so the downstream `PaymentTransaction` state machine never has to reason about replay itself.

12.4 Payment Transaction State Machine

```
stateDiagram-v2
    [*] --> pending: initiate()
    pending --> succeeded: verified webhook OR reconciliation poll
    pending --> failed: verified failure webhook
    pending --> expired: reconciliation timeout (24h, scheduled sweep)
    succeeded --> [*]
    failed --> [*]
    expired --> [*]
```

Transitions are enforced by a dedicated `PaymentTransaction::transitionTo()` method (not a bare `update(['status' => ...])` anywhere in the codebase) that rejects invalid transitions (e.g., `succeeded` → `pending`) — a defensive measure against a delayed/out-of-order webhook overwriting a transaction that a faster reconciliation poll already resolved.

12.5 Reconciliation

A scheduled command (`payments:reconcile-pending`, every 15 minutes, Part 36) polls each provider's transaction-status endpoint for every `PaymentTransaction` still `pending` after 5 minutes — closing the gap where a webhook is lost (provider outage, misconfigured endpoint) without ever polling *before* giving the webhook a fair chance to arrive first, which would create needless API load against the provider.

12.6 Automated Monthly Billing Cycle

`billing:process-subscription-renewals` (scheduled daily, Part 36) finds subscriptions due for renewal, creates the `Invoice`, and initiates a payment via the tenant's saved provider/method — reusing the exact `PaymentProvider` interface and idempotency-key discipline from §12.1, so a renewal that runs twice (a scheduler double-fire, a manual re-trigger during an incident) cannot double-charge a tenant.

Part 13 — Queues, Redis & Messaging

13.1 Why Async at All

Unchanged rationale from the original: payslip PDF generation, bulk email/SMS/push, webhook delivery, payment reconciliation, and accounting sync must never run on the HTTP request thread that triggered them — a payroll run for a 500-employee company cannot make the HTTP client wait for 500 PDFs to render.

13.2 Queue Topology

high	– time-sensitive, small payload (2FA codes, password resets)
default	– general-purpose, low volume
payroll	– payroll-run post-processing (payslip generation fan-out)
notifications	– in-app notification writes
email	– outbound email delivery
sms	– outbound SMS delivery
webhooks	– outbound webhook delivery to tenant-registered endpoints
reports	– compliance report generation (dompdf, lighter weight)
exports	– bulk data exports (CSV/XLSX)
integrations	– accounting-platform sync (OAuth-bound, rate-limited by the external API)

Horizon config assigns worker counts per queue, weighted toward `payroll` and `email` during typical end-of-month traffic patterns, with `auto` balancing so idle capacity on `sms` / `exports` gets redirected to `payroll` under burst load:

```
// config/horizon.php (excerpt)
'environments' => [
  'production' => [
    'supervisor-payroll' => [
      'connection' => 'redis', 'queue' => ['payroll'],
      'balance' => 'auto', 'minProcesses' => 2, 'maxProcesses' => 20,
      'tries' => 3, 'timeout' => 120,
    ],
    'supervisor-email' => [
      'connection' => 'redis', 'queue' => ['email'],
      'balance' => 'auto', 'minProcesses' => 2, 'maxProcesses' => 10,
      'tries' => 5, 'backoff' => [10, 60, 300],
    ],
    'supervisor-webhooks' => [
      'connection' => 'redis', 'queue' => ['webhooks'],
      'balance' => 'auto', 'minProcesses' => 1, 'maxProcesses' => 8,
      'tries' => 6, 'backoff' => [5, 30, 120, 600, 3600, 21600], // up to 6h exponential backoff
    ],
  ],
],
```

13.3 Retry, Backoff, and Dead-Letter Handling

Every `ShouldQueue` job declares `$tries` and `backoff()` explicitly rather than relying on Horizon defaults — matching the original's insistence on explicit-per-job retry policy since notification delivery and payment webhook processing have genuinely different acceptable-failure profiles:

```
final class SendPayslipEmail implements ShouldQueue
{
    public $tries = 5;
    public $maxExceptions = 3;

    public function backoff(): array { return [30, 120, 600, 1800, 7200]; }

    public function failed(Throwable $exception): void
    {
        // After exhausting retries: mark the specific payslip delivery as failed (visible on
        // the payroll-run detail page for HR to retry manually) — never silently drop it.
        PayslipDelivery::where('payroll_entry_id', $this->payrollEntryId)
            ->update(['status' => 'failed', 'failure_reason' => $exception->getMessage()]);
    }
}
```

Jobs that exhaust retries land in Laravel's `failed_jobs` table automatically (Horizon's failed-job UI), which functions as the dead-letter queue — visible, retrievable individually or in bulk, never silently discarded.

13.4 Job Uniqueness and Idempotency

Jobs that must not run twice concurrently for the same target implement `ShouldBeUnique`:

```
final class GeneratePayslip implements ShouldQueue, ShouldBeUnique
{
    public function uniqueness(): string { return $this->payrollEntryId; }
    public function uniqueFor(): int { return 3600; } // lock TTL — released early on completion regardless
}
```

Backed by a Redis lock (`WithoutOverlapping` middleware achieves the same effect for jobs keyed by a runtime value rather than a constructor property). This closes the same class of “duplicate dispatch from a retried HTTP request” bug the `input_hash` uniqueness constraint closes at the payroll-run level (Part 9 §9.9) — the two mechanisms exist at different layers (queue dispatch vs. database write) and are both necessary.

13.5 Event-Driven Fan-Out

```
graph TD
    A[PayrollRunCompleted event] --> B[GeneratePayslips listener]
    A --> C[SendPayrollCompletionNotifications listener]
    A --> D[DispatchOutboundWebhooks listener]
    A --> E[SyncPayrollToAccounting listener]
    B --> F[one GeneratePayslip job PER payroll_entry]
    F --> G[one payslip PDF in object storage]
    G --> H[one SendPayslipEmail job PER entry]
```

Each listener is itself queued (`implements ShouldQueue` on the listener, not just the event) so a slow accounting-sync API call cannot delay payslip generation, and one listener's failure cannot cascade into the others — they are independent queue entries, not a synchronous chain.

When to use which mechanism (per Part 6 §6.4, restated in context):

Situation	Mechanism
The payroll-run command needs the freshly-created <code>PayrollRun</code> back synchronously	Direct method call within the command (§10.2)
Something should happen <i>because</i> a payroll run completed, but the run itself doesn't need to wait or know about it	<code>PayrollRunCompleted</code> domain event → queued listeners
Something should happen on a schedule, independent of any user action	Scheduled Artisan command (Part 36)

13.6 Payslip Generation: Job Granularity

Carried over verbatim from the original's explicit requirement — **never generate payslips synchronously in the HTTP request**, and never as one giant job for the whole run:

```
PayrollRunCompleted
↓
GeneratePayslips listener: dispatch ONE GeneratePayslip job PER payroll_entry (not one job for the whole run)
↓
Each GeneratePayslip job: render Blade template → Browsershot PDF → Storage::disk('s3')->put()
↓
Each job independently dispatches SendPayslipEmail (and SMS/push per notification preferences)
```

One-job-per-payslip means a single employee's PDF-render failure (a malformed name breaking template rendering, an object-storage transient error) retries and fails in isolation — it never blocks or duplicates the other 499 employees' payslips in the same run, and Horizon's per-job retry/backoff (§13.3) applies at exactly the right granularity.

13.7 The Outbox Pattern — Applied Narrowly

```
DB::transaction(function () {
    $run->update(['status' => 'completed']);
    // naive: dispatch(new NotifyPayrollComplete($run));
});
```

The naive version above has a failure window: if the process crashes *after* `DB::commit()` but the dispatch call never executes (or Redis is briefly unreachable at that exact moment), the payroll run is durably marked complete but no downstream notification, payslip, or webhook ever fires — a silent, hard-to-detect gap for a financially significant event.

For the two events where this gap is unacceptable — **payroll-run completion** and **payment settlement** — the write and the “intent to notify” are committed in the *same* database transaction, via an outbox table, and a separate dispatcher polls/publishes from there:

```
DB Transaction
+-- UPDATE payroll_runs SET status = 'completed'
+-- INSERT INTO outbox_events (event_type, payload, dispatched_at = NULL)
↓
OutboxDispatcher (scheduled command, runs every few seconds)
↓
SELECT ... WHERE dispatched_at IS NULL FOR UPDATE SKIP LOCKED
↓
event(PayrollRunCompleted) → mark dispatched_at = now()
```

`FOR UPDATE SKIP LOCKED` lets multiple dispatcher instances run concurrently without double-publishing the same outbox row — a standard Postgres work-queue pattern. Every **other** event in the system (notification-preference-driven fan-out, accounting sync retries, non-financial audit events) tolerates the small, transient at-least-once gap of a direct post-commit `dispatch()` call and does **not** use the outbox — applying it everywhere would add a polling indirection layer to events where an occasionally-missed, easily-retried notification has no real consequence, which is complexity without a corresponding benefit (§2.2 “Why an Outbox Pattern”).

Part 14 — Notifications

14.1 Provider Abstraction

```
// app/Domain/Notifications/Contracts/NotificationChannel.php — Laravel's native Notification system
// already provides exactly this abstraction; PayrollFiti implements custom channels rather than
// reinventing the interface the original hand-rolled.
final class WhatsAppOrSmsChannel { /* implements Laravel's Notification "via" channel contract */ }
```

Laravel's built-in `Notification` class + custom channel classes for SMS (Africa's Talking, matching the original) and Web Push (§Part 22) replace the original's hand-rolled `NotificationService` dispatcher one-for-one — this is a case where the framework's native primitive is already the abstraction the original built from scratch:

```
final class PayrollRunCompletedNotification extends Notification implements ShouldQueue
{
    public function via(object $notifiable): array
    {
        return array_filter([
            'database', // in-app notification center, §18.1-equivalent
            $notifiable->wantsEmail('payroll') ? 'mail' : null,
            $notifiable->wantsSms('payroll') ? AfricasTalkingChannel::class : null,
        ]);
    }

    public function toMail(object $notifiable): MailMessage { /* ... */ }
    public function toDatabase(object $notifiable): array { /* ... */ }
}
```

Every channel dispatch is queued (`ShouldQueue` on the notification class) — no expensive delivery ever happens inline, matching Part 13's blanket rule. Delivery status (`sent`, `failed`, provider message ID) is recorded per channel per notification for the observability requirements of Part 27.

Part 15 — HR Features (Leave, Loans, Attendance, Documents)

These modules carry over as direct functional equivalents — CRUD-plus-workflow domains with no framework-specific redesign beyond living under `Domain/Leave`, `Domain/Loans`, `Domain/Attendance`, `Domain/Documents` per Part 6's structure and inheriting tenancy (Part 8) and authorization (Part 7) for free via `BelongsToTenant` and per-model Policies.

15.1 Leave Management

`LeaveType`, `LeaveBalance`, `LeaveRequest` — approval workflow modeled as an explicit state machine (`pending` → `approved|rejected`, `approved` → `cancelled` within a grace window) enforced the same way as `Payment Transactions` (§12.4): a `transitionTo()` method, never a bare status `update()`. Monthly accrual runs via a scheduled command (`leave:accrue-monthly`, Part 36), idempotent per employee per month (a `leave_accruals` ledger table with a `(employee_id, period)` unique constraint prevents double-accrual on a scheduler double-fire, the same idempotency discipline applied everywhere in this system).

15.2 Loans & Advances

`Loan`, `LoanRepayment` — repayments are generated as an immutable schedule at loan approval time (never recalculated in place), and each period's payroll run deducts the scheduled installment via a `voluntaryDeductions` entry fed into the payroll engine (§9.6) — loan repayment amounts flow **into** the payroll calculation as input, never computed by a second, parallel deduction path outside the engine, preserving the single-source-of-truth principle for anything that touches net pay.

15.3 Attendance

`AttendanceRecord` (present/absent/leave per day) feeds proration inputs for hourly/daily-rate employees where applicable; for salaried employees it remains informational/compliance-adjacent rather than a payroll input, matching the original's scope.

15.4 Employee Documents

Uploaded via `Storage::disk('s3')` with tenant-prefixed keys (Part 8 §8.5) and short-lived signed URLs for download — never a public bucket, never a direct-to-client-visible permanent URL, for the same PII-handling reasons as field-level encryption (Part 7 §7.7).

Part 16 — Employee Onboarding

Onboarding is modeled as `OnboardingTask` rows generated from a per-company checklist template at employee creation, tracked to completion (document upload, contract e-signature acknowledgment, benefits enrollment) — unchanged in shape from the original. The invitation-to-portal-access flow is detailed fully in Part 7 §7.4; onboarding tasks become visible to the employee immediately after invitation acceptance, gated by the same Policy-based authorization as every other employee-portal resource (an employee can only see and complete *their own* onboarding tasks — `OnboardingTaskPolicy::view()` checks `$task->employee->user_id === $user->id`).

Part 17 — 2FA & Session Management

17.1 TOTP Setup

Three-call flow, unchanged from the original's explicit design ("Three Calls, Not One" — generate a secret without activating it, verify a code against the pending secret, only then persist it as active, so a user who abandons setup midway never ends up locked out by a half-configured factor):

```
POST /account/2fa/setup      → generates TOTP secret + QR code, stored PENDING (not yet enforced)
POST /account/2fa/verify    → user submits a code generated from an authenticator app; on success,
                             secret is marked ACTIVE and 10 recovery codes are generated (shown once)
POST /login/2fa             → second step of login (§7.2) once 2FA is ACTIVE for the account
```

Recovery codes are hashed at rest (bcrypt, one-way, same discipline as passwords — never reversibly encrypted, since they're single-use credentials, not data that needs to be redisplayed) and each is invalidated on use.

17.2 Sessions as Multi-Device, Individually Revocable

Detailed in Part 7 §7.5: Sanctum's native multi-token model replaces the original's single-shared-token-slot design outright, rather than patching it — each login creates a distinct session (Redis-backed), listed and individually revocable at `GET /account/sessions` / `DELETE /account/sessions/{id}`, with a `DELETE /account/sessions` ("log out everywhere else") convenience endpoint that revokes all sessions except the requester's current one.

Part 18 — API Keys & Webhooks

18.1 API Keys — Scoped Machine Credentials

Sanctum personal access tokens (§2.4, §7.1) serve directly as PayrollFiti's public-API credential: `abilities()` provide the scoping (`employees:read`, `payroll:read`, `webhooks:manage`), `last_used_at` gives visibility into whether a key is still active, and revocation is a single `$token->delete()`. The raw token is shown exactly once at creation time (Laravel's standard Sanctum behavior) and only its hash is ever persisted — the same never-store-secrets-in-plaintext discipline as invitation tokens (§7.4) and password-reset tokens (§7.3).

18.2 Outbound Webhooks

`WebhookEndpoint` (tenant-registered URL + subscribed event types) and `WebhookDeliveryLog` (one row per delivery attempt) carry over unchanged. Delivery is queued (`webhooks` queue, §13.2) with HMAC request signing so the receiving endpoint can verify authenticity:

```
final class DeliverWebhook implements ShouldQueue
{
    public $tries = 6;
    public function backoff(): array { return [5, 30, 120, 600, 3600, 21600]; }

    public function handle(): void
    {
        $payload = json_encode($this->event);
        $signature = hash_hmac('sha256', $payload, $this->endpoint->signing_secret);

        $response = Http::withHeaders(['X-PayrollFiti-Signature' => $signature])
            ->timeout(10)
            ->post($this->endpoint->url, $this->event);

        WebhookDeliveryLog::create([
            'webhook_endpoint_id' => $this->endpoint->id,
            'response_status' => $response->status(),
            'attempt' => $this->attempts(),
        ]);

        $response->throw(); // triggers Laravel's retry/backoff on non-2xx, per $tries above
    }
}
```

Endpoints that fail every retry for a sustained period (a configurable threshold, e.g. 20 consecutive failed deliveries) are automatically flagged `disabled` with a dashboard alert to the tenant admin, rather than continuing to retry indefinitely against a dead URL.

18.3 The Audit Log

`AuditLog` carries over unchanged as a cross-cutting concern: an `Auditable` trait + model observer records create/update/delete on every domain model marked auditable, capturing actor, tenant, before/after diff (redacted for encrypted fields, §7.7), and IP. The original's own account of “a real leak it caught” — a bug the audit trail surfaced in production — is exactly the kind of value this mechanism is designed to provide, and nothing about the redesign weakens it: the audit write happens inside the same request-scoped tenant context as everything else (Part 8), so audit rows themselves are tenant-isolated by the identical two-layer mechanism.

Part 19 — Accounting Integrations

19.1 OAuth-Integrated Platforms

Xero, QuickBooks, and Zoho Books integrate via each platform's OAuth2 authorization-code flow — this is the one place in the system PayrollFiti is an OAuth **client**, not a provider (distinct from Sanctum's role as PayrollFiti's own inbound auth, §2.4). Laravel Socialite (extended with community/custom providers for Xero and QuickBooks, which aren't in Socialite's default set) handles the authorization dance; refresh tokens are stored encrypted (`encrypted` cast, §7.7) per tenant per platform.

19.2 Three Platforms, One Interface

```
interface AccountingProvider
{
    public function syncPayrollRun(PayrollRun $run): SyncResult;
    public function syncInvoice(Invoice $invoice): SyncResult;
}
```

`SyncPayrollToAccounting` (Part 13 §13.5's listener) resolves the tenant's configured provider and calls this interface — adding a fourth accounting platform means one new implementation class and a registry entry, the same extension pattern used throughout this guide (payment providers, country rule sets, compliance reports).

19.3 Wired Into Billing and Payroll

Sync is best-effort and asynchronous (the `integrations` queue, §13.2, deliberately isolated from `payroll/email` so a slow or rate-limited third-party accounting API can never delay payslip delivery), with failures surfaced on a per-tenant integration-health dashboard rather than silently retried forever — matching the original's explicit acknowledgment that “webhook dispatch and accounting sync are best-effort” after a payment or payroll event, now made visible operationally rather than left implicit.

Part 20 — Next.js Frontend Architecture

20.1 Directory Structure

```
apps/web/
├── app/
│   ├── (marketing)/      # /, /pricing, /reviews, /contact, /blog – Server Components, SSG/ISR
│   ├── (auth)/           # /login, /signup, /forgot-password, /invitations/[token]
│   └── (dashboard)/      # /dashboard, /employees, /payroll, /leave, /loans, /billing – Client
Components + TanStack Query
├── components/           # shared, mostly presentational (shadcn/ui-based)
├── features/
│   ├── payroll/          # feature-local components, hooks, API bindings
│   ├── employees/
│   ├── leave/
│   ├── loans/
│   └── billing/
├── lib/
│   ├── api/              # typed client, generated from OpenAPI (§43) + hand-written hooks
│   ├── auth/             # Sanctum cookie/CSRF handling, session context
│   └── validation/       # Zod schemas mirroring Laravel Form Requests
├── hooks/
├── stores/               # minimal client-only UI state (Zustand) – never server state, that's
TanStack Query's job
├── types/                # generated from OpenAPI
└── tests/
```

This mirrors the original's feature-oriented layout exactly — the frontend's architecture is language-agnostic to the backend rewrite; what changes is only *what* `lib/api/` talks to.

20.2 Rendering Strategy per Route Type

Route type	Strategy	Why
Marketing pages (/ , /pricing , /reviews)	Server Components, static generation	No auth, content changes rarely, SEO-critical
Blog (/blog/[slug])	Server Components, ISR (revalidate on Sanity webhook, Part 23)	Content-managed, needs fresh content without a full rebuild
Public payroll calculator (/calculator)	Client Component	Genuinely interactive, calls the public API endpoint (§10.5) per keystroke/submit
Authenticated dashboard (/dashboard , /employees , ...)	Client Components + TanStack Query	Highly interactive, per-user data, no SEO need, benefits from client-side cache/optimistic updates
Mutations (create employee, run payroll, approve loan)	Typed API client + TanStack Query mutations	Consistent loading/error/optimistic-update handling across every mutating action

Server Actions are used narrowly — only for simple, non-authenticated-dashboard form submissions where a full client-side mutation hook would be overkill (e.g., the marketing site's contact form) — the authenticated app's data layer is deliberately **not** built on Server Actions, because TanStack Query's client-side cache (shared across every dashboard page, invalidated declaratively) is a better fit for a dashboard where the same data (e.g., employee list) is read and mutated from many different screens.

20.3 Business Logic Stays in Laravel

No payroll calculation, tax logic, or authorization decision is ever duplicated in the frontend. The frontend's only "logic" is: (1) client-side Zod validation as a UX nicety (instant feedback before a round trip), always re-validated server-side by the matching Form Request; and (2) conditionally rendering UI based on the `permissions` array from `/user` (Part 7 §7.6) — never the authorization *decision* itself, which is always re-checked server-side on every mutating request regardless of what the UI allowed the user to attempt.

Part 21 — Frontend Application

21.1 The API Client — Token Refresh Is Transparent (Reframed for Sanctum)

The original's “token refresh transparent to every caller” requirement doesn't disappear under Sanctum's cookie model — it simplifies. There is no access/refresh token pair to juggle client-side; the browser sends the session cookie automatically on every request. What the client *does* still need to handle transparently:

```
// lib/api/client.ts
export const apiClient = async <T>(path: string, init?: RequestInit): Promise<T> => {
  const csrfToken = getCookie('XSRF-TOKEN');
  const response = await fetch(`${API_URL}${path}`, {
    ...init,
    credentials: 'include', // send the session cookie
    headers: { ...init?.headers, 'X-XSRF-TOKEN': csrfToken, Accept: 'application/json' },
  });

  if (response.status === 401) {
    // Session expired/revoked mid-app-use — redirect to login, preserving intended destination.
    window.location.href = `/login?redirect=${encodeURIComponent(window.location.pathname)}`;
    throw new UnauthorizedError();
  }

  if (!response.ok) throw await ApiError.fromResponse(response);
  return response.json();
};
```

401 handling is centralized here exactly once — every feature's TanStack Query hook goes through this client, so no individual page implements its own “what if the session expired” logic, matching the original's design intent even though the underlying credential mechanism changed.

21.2 Data Fetching — TanStack Query, No Hand-Rolled `useEffect`

```
// features/payroll/hooks/usePayrollRuns.ts
export const usePayrollRuns = (companyId: string) =>
  useQuery({
    queryKey: ['payroll-runs', companyId],
    queryFn: () => apiClient<PayrollRun[]>(`/v1/companies/${companyId}/payroll/runs`),
  });

export const useRunPayroll = (companyId: string) => {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: (input: RunPayrollInput) =>
      apiClient<PayrollRun>(`/v1/companies/${companyId}/payroll/runs`, { method: 'POST', body: JSON.stringify(input) }),
    onSuccess: () => queryClient.invalidateQueries({ queryKey: ['payroll-runs', companyId] }),
  });
};
```

21.3 Template: Payroll Run List → Detail

Unchanged in structure from the original — a list page (`usePayrollRuns`) linking to a detail page (`usePayrollRun(runId)`) showing per-employee entries, with the “approve” mutation gated client-side by the `permissions` array (§20.3) and re-checked server-side by `PayrollRunPolicy::approve()` (Part 7 §7.6).

21.4 Employee Self-Service Portal

The same Next.js app, same route group structure, scoped by role: an `employee-role` user's `/dashboard` renders their own payslips, leave balance, and loan status rather than the HR/admin views — driven entirely by the `roles / permissions` returned from `/user`, not a separate frontend deployment.

Part 22 — PWA & Push Notifications

22.1 Service Worker & Offline Support

Servist (unchanged from the original — a frontend-only concern, independent of backend language) provides the service worker, precaching, and an `/offline` fallback so the employee self-service portal can show a cached payslip with no connection.

22.2 Web Push — Backend

Laravel ships first-class Web Push support via `laravel-notification-channels/webpush`, replacing the original's hand-rolled VAPID/push-subscription handling with a maintained package built specifically for Laravel's `Notification` system (§14.1) — subscriptions are stored per-user, and any notification class that lists `WebPushChannel::class` in its `via()` array delivers to every registered device automatically:

```
final class PayrollRunCompletedNotification extends Notification
{
    public function via(object $notifiable): array
    {
        return array_filter([..., $notifiable->pushSubscriptions()->exists() ? WebPushChannel::class : null]);
    }

    public function toWebPush(object $notifiable, $notification): WebPushMessage
    {
        return (new WebPushMessage)->title('Payroll run completed')->icon('/icon-192.png')->action('View', 'view_payroll');
    }
}
```

22.3 Install Prompt

Frontend-only — the standard `beforeinstallprompt` event handling, unchanged from the original.

Part 23 — Blog & CMS

23.1 Why a Headless Store, Not a Custom Admin

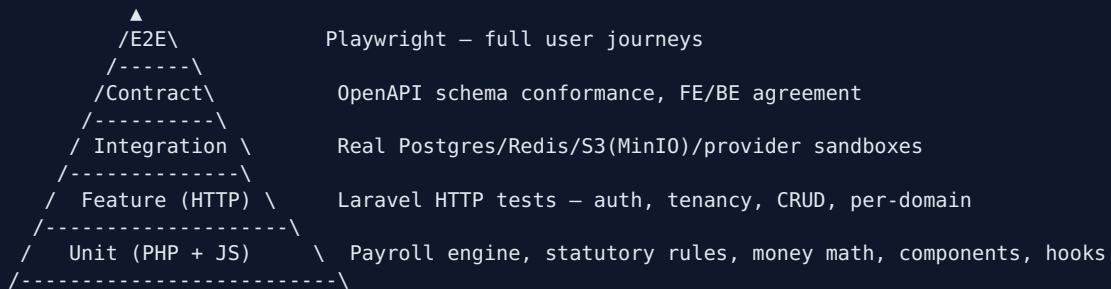
Sanity (unchanged from the original) remains the right choice here specifically *because* it's decoupled from the backend language — the blog's content model, admin dashboard, and asset pipeline have zero dependency on whether the API is NestJS or Laravel. The public blog (`/blog/[slug]`, Server Components + ISR) fetches directly from Sanity's API, not through the Laravel backend at all, for the same reason the original chose this shape: blog content isn't tenant data and doesn't need PayrollFiti's auth/tenancy machinery in the request path.

23.2 The One Integration Point: Restricting Blog Management to One Tenant

The original's gate — “restricting blog management to one tenant” (PayrollFiti's own marketing team, not customer tenants) — is enforced the same way: Sanity Studio's own access control (project-level members), with no PayrollFiti backend involvement at all. This is a case where the cleanest Laravel-specific answer is *no* integration — the original correctly kept this system decoupled, and the redesign preserves that decoupling rather than looking for a reason to route it through Laravel.

Part 24 — Testing Strategy

24.1 The Pyramid



24.2 Unit Tests (Backend)

Pest, run without booting the full HTTP kernel where possible. The payroll engine's tests (Part 9 §9.10) are the canonical example: pure input/output assertions, no database. Domain services with light framework touch (e.g., a repository interface's in-memory fake) are tested the same way.

24.3 Feature/API Tests

```
// tests/Feature/Payroll/RunPayrollTest.php
it('rejects a second concurrent run for the same period with the same result', function () {
    $company = Company::factory()->create(['country' => 'KE']);
    $admin = User::factory()->admin()->for($company->tenant)->create();
    Employee::factory()->count(3)->for($company)->create();

    [$first, $second] = collect([1, 2])->map(fn () =>
        $this->actingAs($admin)->postJson("/api/v1/companies/{ $company->id }/payroll/runs", [
            'period_start' => '2026-01-01', 'period_end' => '2026-01-31', 'pay_date' => '2026-02-01',
        ])
    );

    expect($first->json('data.id'))->toBe($second->json('data.id'))
        ->and(PayrollRun::count()->toBe(1)); // idempotency held, Part 9 §9.9 / Part 10 §10.4
});

it('returns 404, not 403, for a company belonging to another tenant', function () {
    $otherTenantCompany = Company::factory()->create();
    $this->actingAs(User::factory()->admin()->create())
        ->getJson("/api/v1/companies/{ $otherTenantCompany->id }")
        ->assertNotFound(); // Part 8 tenancy behavior — never confirm existence to a non-owner
});
```

Every domain (employees, payroll, leave, loans, documents, billing, webhooks) has a feature test suite covering: happy path, authorization denial, tenant-isolation denial, and validation failure — the four cases the original called out as mandatory per module.

24.4 Integration Tests

Run against real Postgres, Redis, and MinIO (S3-compatible) in CI (Part 34) — not mocks — specifically for the code paths mocks would give false confidence about: RLS policy behavior (Part 8 §8.2), queue job serialization round-trips, and object-storage signed URLs. Payment-provider sandboxes (Paystack test mode, Daraja sandbox) are used for the payment-flow integration tests, gated behind CI secrets and run less frequently (nightly, not per-PR) to avoid rate-limiting the sandbox accounts.

24.5 Contract Tests

An OpenAPI spec is generated from Laravel routes + Form Requests + Resources (`dedoc/scramble` or an equivalent introspection package). A CI step (Part 34) validates every Feature test's actual HTTP response against this spec's schema (`spectator` -style assertion), and the frontend generates its TypeScript API types from the same spec (`openapi-typescript`) — this is what keeps Next.js and Laravel in agreement without hand-duplicated interfaces, replacing the original's single-language shared-types package (§3.1's noted structural change).

24.6 E2E Tests (Playwright)

```
Admin signup → create company → create employee → configure salary
  → run payroll → approve payroll → generate payslip
  → employee logs in → employee downloads payslip
```

Plus: leave request approval, loan approval, password reset, 2FA enrollment + login, a full payment flow against sandbox providers, notification delivery (asserted via Mailpit's API in the test environment), tenant-isolation (two tenants' data never crosses in the UI), and PWA install/offline behavior.

24.7 Database Testing

Migration up/down round-trip tested in CI on every PR. Model factories provide realistic, country-varied test data (a `KenyaEmployeeFactory` state, etc.). The concurrency test in §24.3 is the canonical race-condition test the original explicitly required.

Part 25 — Security Testing

Layer	Tool	Role
Dependency CVEs (PHP + JS)	GitHub Dependabot	Automated PRs for vulnerable <code>composer.lock</code> / <code>pnpm-lock.yaml</code> entries
SAST (static analysis for insecure patterns)	Semgrep	Custom + community rule sets — catches raw SQL string interpolation, missing <code>Gate::authorize()</code> on state-changing controllers, <code>withoutGlobalScope</code> outside the sanctioned helper (Part 8 §8.3)
Container image CVEs	Trivy	Scans built Docker images in CI before push (Part 34)
Secret scanning	GitLeaks	Pre-commit hook + CI — blocks a committed <code>.env</code> , API key, or private key before it reaches history
DAST	OWASP ZAP	Scheduled scan against staging (not production) — SQLi/XSS/header-misconfiguration sweep

Additional PayrollFiti-specific security tests, beyond the generic tools above: authorization-matrix tests (every role × every endpoint, asserting exactly the expected allow/deny), the tenant-isolation Feature tests from §24.3 run as a dedicated suite tagged `@tenancy` (so they can be run in isolation as a fast pre-merge gate), webhook-signature verification tests (Part 12 §12.3, Part 18 §18.2 — reject unsigned/mis-signed payloads), rate-limit tests (login throttling, public-calculator throttling, §10.5), and session-security tests (revoked session immediately rejects subsequent requests, no cookie replay after logout).

Part 26 — Load Testing

26.1 k6 Scenarios

```
// load/payroll-run.js
import http from 'k6/http';
import { check } from 'k6';

export const options = { vus: 20, duration: '2m' };

export default function () {
  const res = http.post(`${__ENV.API_URL}/api/v1/companies/${__ENV.COMPANY_ID}/payroll/runs`,
    JSON.stringify({ period_start: '2026-02-01', period_end: '2026-02-28', pay_date: '2026-03-01' }),
    { headers: { 'Content-Type': 'application/json', Authorization: `Bearer ${__ENV.API_TOKEN}` } });
  check(res, { 'status is 201 or 200 (idempotent)': (r) => [200, 201].includes(r.status) });
}
```

Scenarios cover: login, employee listing (paginated, realistic dataset sizes per §37's tenant-size table), payroll-run submission and detail retrieval, payslip download (signed URL generation), notification-center polling, and inbound webhook processing throughput.

26.2 Targets (documented, not invented)

Endpoint class	Target	How verified
Simple CRUD reads (employee list, leave balance)	p95 < 300ms	k6 scenario + Grafana dashboard correlation during the test window
Complex reads (payroll run detail with entries, compliance report data)	p95 < 1s	Same
Payroll run submission (compute-bound, scales with employee count)	Documented per-employee-count benchmark, not a single fixed target — e.g. “~15ms of engine compute per employee on reference hardware,” re-measured whenever the engine changes	Dedicated benchmark script, run in CI on <code>Domain/Payroll/Engine</code> changes, not asserted as a hard gate but tracked over time to catch regressions
Queue processing latency (<code>payroll</code> queue, time from job dispatch to completion)	p95 < 5s per payslip job under normal worker counts	Horizon metrics + Prometheus histogram (Part 28)

These are working targets validated by the k6 runs above and Prometheus histograms in staging under simulated load — not guarantees extrapolated without measurement, per the original brief's explicit instruction not to invent unrealistic numbers.

Part 27 — Observability: Structured Logging

27.1 Structured JSON Logs

```
// config/logging.php — 'stack' channel includes a custom formatter
'formatter' => Monolog\Formatter\JsonFormatter::class,
```

```
{
  "timestamp": "2026-09-09T10:15:32Z",
  "level": "error",
  "service": "payrollfiti-api",
  "environment": "production",
  "request_id": "req_01J...",
  "correlation_id": "corr_01J...",
  "tenant_id": "ten_9f3...",
  "user_id": "usr_2ab...",
  "job_id": null,
  "route": "POST /api/v1/companies/{company}/payroll/runs",
  "duration_ms": 842,
  "message": "Payroll processing failed: unsupported country rule version"
}
```

`request_id` / `correlation_id` are generated (or propagated from an inbound `X-Correlation-Id` header, set by Nginx/Next.js) in a `AssignCorrelationId` middleware, registered first in the HTTP kernel pipeline, and injected into every log line via a bound `Illuminate\Log\Context` (Laravel 11's native contextual-logging feature) — so a job dispatched mid-request inherits the same correlation ID automatically when logging from `handle()`, tracing a single logical operation across the HTTP request and every queue job it spawned.

Never logged, enforced by a redaction processor applied to every log record before it's written: passwords, access tokens, refresh tokens/session cookies, payment provider secrets (webhook signing keys, API secrets), full bank account/national ID numbers (only a masked last-4 form), and any field flagged `encrypted` on its Eloquent model (§7.7) — reflected automatically by a `RedactsSensitiveAttributes` trait. Log context helpers consult before serializing a model into a log line.

Part 28 — Prometheus & Grafana

28.1 Metrics Exposed

`spatie/laravel-prometheus` (or an equivalent PHP Prometheus client) exposes `/metrics`, scraped by Prometheus, restricted to internal network access only (never public, Part 31's Nginx config denies external access to this path):

<code>http_request_duration_seconds{route, method, status}</code>	– histogram
<code>http_requests_total{route, method, status}</code>	– counter
<code>queue_job_duration_seconds{queue, job}</code>	– histogram
<code>queue_jobs_failed_total{queue, job}</code>	– counter
<code>queue_depth{queue}</code>	– gauge (Horizon metrics API)
<code>payroll_run_duration_seconds{country}</code>	– histogram
<code>payroll_runs_total{status}</code>	– counter
<code>db_connections_active / db_connections_max</code>	– gauge
<code>redis_connected_clients / redis_used_memory_bytes</code>	– gauge
<code>cache_hit_ratio</code>	– gauge
<code>webhook_delivery_failures_total{endpoint_id}</code>	– counter
<code>payment_transactions_total{provider, status}</code>	– counter

28.2 Grafana Dashboards (provisioned as code, versioned in `infra/grafana/dashboards/`)

Dashboard	Key panels
Application	requests/sec, error rate, p50/p95/p99 latency by route
Payroll	runs/day, processing duration distribution, success vs. failed runs, employees processed/run
Queues	per-queue depth, failed-job count, throughput, Horizon worker utilization
Database	active connections vs. max, query latency, slow-query count, table/index bloat, replication lag
Redis	memory usage, hit/miss ratio, connected clients, commands/sec, eviction count
Payments	success rate by provider, webhook failure rate, reconciliation backlog size

Part 29 — Sentry & Error Tracking

Sentry (DSN-gated, off by default in local dev — unchanged from the original's explicit “never phones home locally” requirement) captures unhandled Laravel exceptions (via `sentry/sentry-laravel`'s exception handler integration), Next.js client and server errors, and queue-job failures — each tagged with the same `tenant_id/correlation_id/release` context as the structured logs in Part 27, so a Sentry issue and its corresponding log lines can be cross-referenced by correlation ID during an incident. **Why both Sentry and Prometheus:** Prometheus answers “*is the error rate elevated right now*” (aggregate, cheap, always-on); Sentry answers “*what exactly broke, for which tenant, on which release, and how many distinct users hit it*” (detailed, sampled, queried during investigation) — replacing either with the other loses a genuinely different question each is built to answer.

Part 30 — OpenTelemetry & Distributed Tracing

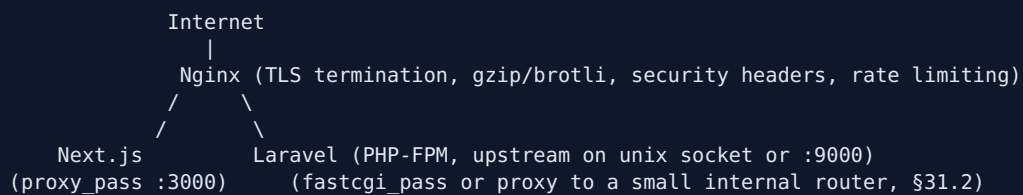
Introduced at the Phase 1 → Phase 2 boundary (§2.6), instrumented via the OpenTelemetry PHP SDK's Laravel auto-instrumentation package plus the Next.js OTel integration, propagating a `traceparent` header end to end:

```
Browser → Next.js (span) → Nginx (propagates header, does not span) → Laravel API (span)
  → PostgreSQL query (span) → Redis (span) → Queue job dispatch (span, linked not nested – async)
  → Queue worker pickup (new trace root, linked to dispatch span via job payload's trace context)
  → External provider HTTP call (span)
```

Queue-crossing traces are **linked**, not nested, because the dispatching request completes before the job executes — OpenTelemetry's span-links feature (not parent-child) correctly represents “this job was caused by that request” without falsely extending the request's duration to include async work that happens after the response was already sent. Traces export to an OpenTelemetry Collector, which forwards to Grafana Tempo (kept in the same Prometheus/Grafana operational family established in Part 28 rather than introducing a third vendor).

Part 31 — Nginx

31.1 Topology



Should Next.js run behind Nginx directly? Yes — Nginx terminates TLS and serves as the single public entry point for both the marketing/app frontend and the API, so the browser only ever sees one origin/certificate, and `SameSite=Lax` cookies (Part 7) work without cross-origin cookie complications. Nginx proxies `/` to the Next.js container and `/api/` to Laravel.

```
# docker/nginx/conf.d/payrollfiti.conf
upstream nextjs { server web:3000; }
upstream laravel_fpm { server api:9000; }

limit_req_zone $binary_remote_addr zone=api_general:10m rate=20r/s;
limit_req_zone $binary_remote_addr zone=login:10m rate=5r/m;

server {
    listen 443 ssl http2;
    server_name app.payrollfiti.com;

    ssl_certificate      /etc/nginx/certs/fullchain.pem;
    ssl_certificate_key  /etc/nginx/certs/privkey.pem;

    add_header X-Frame-Options "DENY" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;
    add_header Strict-Transport-Security "max-age=63072000; includeSubDomains" always;
    add_header Content-Security-Policy "default-src 'self'; img-src 'self' data: https;; script-src 'self'"
always;

    client_max_body_size 20m; # document/payslip upload ceiling

    location /metrics { deny all; } # never exposed publicly – Prometheus scrapes over the internal network
only

    location /api/v1/auth/login {
        limit_req zone=login burst=3 nodelay;
        try_files $uri @laravel;
    }

    location /api/ {
        limit_req zone=api_general burst=40 nodelay;
        try_files $uri @laravel;
    }

    location @laravel {
        fastcgi_pass laravel_fpm;
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME /var/www/html/public/index.php;
        fastcgi_read_timeout 60s;
    }

    location / {
        proxy_pass http://nextjs;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Correlation-Id $request_id; # propagated for Part 27/30 tracing
    }
}
```

Rate limiting is applied at Nginx (coarse, IP-based, cheap, blocks abuse before it reaches PHP-FPM at all) *and* at Laravel's `RateLimiter` (fine-grained, per-user/per-key — Part 7 §7.2's `email + ip` login limiter) — two layers for the same reason tenant isolation and payroll idempotency get two layers throughout this guide: defense in depth, each catching what the other might miss.

WebSockets are not currently used anywhere in PayrollFiti's design (notifications are polled via TanStack Query + delivered via Web Push, Part 22, not a persistent socket connection), so no `Upgrade / Connection` header handling is needed — noted explicitly here as a **Recommended Architecture (new)** decision rather than silently omitted, since it's a reasonable question a reviewer would ask.

Part 32 — Production Docker

32.1 Multi-Stage Dockerfile (PHP)

```
# docker/php/Dockerfile
FROM composer:2 AS vendor
WORKDIR /app
COPY backend/laravel/composer.json backend/laravel/composer.lock ./
RUN composer install --no-dev --no-scripts --no-autoloader --prefer-dist

FROM php:8.3-fpm-alpine AS base
RUN apk add --no-cache postgresql-dev icu-dev oniguruma-dev \
    && docker-php-ext-install pdo_pgsql intl mbstring bcmath opcache
COPY --from=vendor /usr/bin/composer /usr/bin/composer

FROM base AS app
WORKDIR /var/www/html
COPY backend/laravel/ .
COPY --from=vendor /app/vendor ./vendor
RUN composer dump-autoload --optimize --classmap-authoritative \
    && chown -R www-data:www-data storage bootstrap/cache
USER www-data
COPY docker/entrypoint.sh /entrypoint.sh
ENTRYPOINT ["/entrypoint.sh"]
CMD ["php-fpm"]

FROM app AS worker
# Same image, different CMD — Browsershot's headless Chromium only installed here, not in the API image
USER root
RUN apk add --no-cache chromium
USER www-data
CMD ["php", "artisan", "queue:work", "--queue=payroll,default"]
```

Separating `app` (API, PHP-FPM) from `worker` (queue processing) at the Dockerfile-target level — not just at the Compose-service level — means the API image stays lean (no headless Chromium, ~120MB smaller, faster cold start) while worker images carry what payslip generation (§2.5) actually needs.

32.2 Container-per-Process, Not Supervisor-in-Container

Decision	Chosen Approach	Alternative	Why Chosen	Cost/Tradeoff
Process topology	One container per logical process (<code>api</code> , <code>horizon</code> , <code>scheduler</code> , <code>nginx</code> , <code>nextjs</code>)	Supervisor managing multiple processes inside one container	Matches standard container orchestration expectations (health checks, restarts, and resource limits apply per-process, not per-bundle); a worker crash doesn't risk taking the API down in the same container	More containers to define and monitor — accepted because it directly enables Part 37's independent-scaling requirement (Horizon workers scale differently than API instances under load)

`docker-compose.prod.yml` overlays resource limits, replica counts, and removes the bind-mounted source volume from dev's override, relying entirely on the built image — no live-reload machinery in production.

Part 33 — entrypoint.sh

```
#!/bin/sh
# docker/entrypoint.sh
set -e

echo "[entrypoint] Validating required environment variables..."
for var in APP_KEY DB_HOST DB_DATABASE REDIS_HOST; do
    eval "val=\${$var}"
    if [ -z "$val" ]; then echo "[entrypoint] FATAL: $var is not set" >&2; exit 1; fi
done

echo "[entrypoint] Ensuring storage/bootstrap-cache permissions..."
chown -R www-data:www-data storage bootstrap/cache 2>/dev/null || true

if [ "$APP_ENV" = "production" ]; then
    echo "[entrypoint] Caching config, routes, views..."
    php artisan config:cache
    php artisan route:cache
    php artisan view:cache
    php artisan event:cache
fi

# Deliberately NOT running `php artisan migrate` here — see Part 35 for why.

echo "[entrypoint] Starting: $"
exec "$@"
```

Why migrations do not run here: every container built from this image starts via this entrypoint — API, Horizon, scheduler, and every horizontally-scaled replica of each. Running `migrate --force` in the entrypoint means N containers race to migrate simultaneously on every deploy, and a rolling deployment could have old and new containers both trying to migrate against a schema mid-transition. Migrations run exactly once, as a dedicated step in the deployment pipeline, *before* any application container using the new schema starts (Part 35).

Part 34 — CI/CD

34.1 Pull Request Pipeline

```
# .github/workflows/pr.yml
name: PR
on: pull_request

jobs:
  lint-and-analyze:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: shivammathur/setup-php@v2
        with: { php-version: '8.3', tools: composer }
      - run: composer install --prefer-dist --no-progress
      - run: ./vendor/bin/pint --test
      - run: ./vendor/bin/phpstan analyse --memory-limit=1G
      - run: ./vendor/bin/deptrac analyse # Part 6 §6.3 boundary enforcement

  backend-tests:
    runs-on: ubuntu-latest
    services:
      postgres: { image: postgres:16-alpine, env: { POSTGRES_PASSWORD: test }, options: >-
        --health-cmd pg_isready --health-interval 5s --health-timeout 5s --health-retries 10 }
      redis: { image: redis:7-alpine }
    steps:
      - uses: actions/checkout@v4
      - run: composer install --prefer-dist
      - run: cp .env.testing .env && php artisan key:generate
      - run: php artisan migrate --force
      - run: php artisan test --parallel --coverage --min=80

  security-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: returntocorp/semgrep-action@v1
      - uses: gitleaks/gitleaks-action@v2
      - run: composer audit

  frontend-build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
      - run: pnpm install --frozen-lockfile
      - run: pnpm --filter web lint && pnpm --filter web test && pnpm --filter web build

  e2e:
    needs: [backend-tests, frontend-build-and-test]
    runs-on: ubuntu-latest
    steps:
      - run: docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d
      - run: pnpm --filter web exec playwright test

  docker-build-and-scan:
    needs: [backend-tests, security-scan]
    runs-on: ubuntu-latest
    steps:
      - run: docker build -f docker/php/Dockerfile --target app -t payrollfiti-api:${{ github.sha }} .
      - uses: aquasecurity/trivy-action@master
        with: { image-ref: 'payrollfiti-api:${{ github.sha }}', severity: 'CRITICAL,HIGH', exit-code: '1' }
```

34.2 Merge-to-Main Pipeline

```
Merge to main
↓
Build immutable images, tagged with the git SHA (never `:latest` in production)
↓
Push to registry (GHCR)
↓
Deploy to staging (automatic)
↓
Run migration job against staging DB (Part 35)
↓
Smoke tests (health checks + a handful of critical-path Feature/E2E tests against staging)
↓
Manual production-deploy approval gate
↓
Run migration job against production DB (expand/contract-safe, Part 35)
↓
Rolling deploy of application containers, tagged image
↓
Post-deploy health checks (all `/ready` checks passing across new instances)
↓
Automatic rollback to previous image tag if health checks fail within a defined window
```

Every environment (staging, production) is a distinct GitHub Actions **environment** with its own secrets (§38.4) and required reviewers on the production deploy gate — no secret is ever shared across environments, and the production deploy is never fully automatic, matching the original's emphasis on a deliberate, reviewable release step for a payroll system.

Part 35 — Deployment & Migrations

35.1 Why `php artisan migrate --force` Never Runs From Every Container

Restated concretely from Part 33: if migrations ran in the entrypoint, a rolling deploy with 5 API replicas would have up to 5 concurrent `migrate` invocations racing against the same schema — Laravel's migration locking (`migrations` table's lock mechanism) prevents true corruption, but it turns every deploy into an unnecessary race with unpredictable ordering and no clear point to say “the schema is now definitely ready.” Instead:

```
Build → Test → Deploy MIGRATION JOB (one-shot, separate from app containers) → Deploy application → Health check → (rollback if needed)
```

The migration job is a single, one-shot container invocation (`php artisan migrate --force`), run to completion *before* any application container serving the new code starts, with its own failure gate: if the migration job fails, the deployment pipeline stops before touching any running application container.

35.2 Expand/Contract for Backward-Compatible Rolling Deploys

Because old and new application code briefly run side by side during a rolling deploy (some replicas updated, some not yet), every schema change that isn't purely additive follows expand/contract across two releases rather than one:

```
Release N:   ADD new_column (nullable) – old code ignores it, new code can start writing to it
Release N+1: BACKFILL new_column from old_column (a data migration, run as its own step)
Release N+2: Application code fully cut over to new_column; old_column marked deprecated
Release N+3: DROP old_column – only once no running code references it
```

Renaming a column, changing a type, or dropping a `NOT NULL`-constrained column never happens in a single release; each is decomposed into this multi-release sequence, documented per-migration in the PR description so a reviewer can verify the current release is genuinely backward-compatible with the *previous* release's code, which is the actual invariant a rolling deploy requires.

Part 36 — Backups & Disaster Recovery

36.1 PostgreSQL

Continuous WAL archiving to S3 (via `pgbackrest` or a managed provider's native equivalent) gives point-in-time recovery; a full base backup runs nightly. **RPO target: under 5 minutes** (bounded by WAL-shipping interval). **RTO target: under 1 hour** for a full restore-and-verify — both validated by a monthly scheduled restore drill against a scratch environment, not merely assumed from the backup configuration.

36.2 Object Storage

S3 versioning enabled on the payslips/documents bucket (protects against accidental overwrite/delete, not just infrastructure loss) with lifecycle rules transitioning older payslip versions to cold storage rather than deleting them, matching statutory multi-year retention requirements noted in Part 5 §5.5.

36.3 Redis

Redis's data falls into two categories with very different recovery needs: **queue jobs and cache** are safely reconstructable (a lost queue job is caught by the failed-jobs/dead-letter visibility of Part 13 §13.3; cache repopulates on next read) and are *not* backed up — persistence (RDB/AOF) is enabled only to reduce, not eliminate, work lost on a restart. **Sessions** (Part 7 §7.5) are the one Redis-resident data type with real user-facing impact if lost (forced re-login for every active user) but are explicitly *not* treated as durable data requiring backup — an acceptable, documented tradeoff given how cheap re-authentication is compared to the operational cost of treating session storage as a backed-up system of record.

36.4 Scheduled Jobs (Part 6's third cross-domain communication channel)

Schedule	Command	Idempotency guarantee
Every 15 min	<code>payments:reconcile-pending</code>	Reads-then-transitions via the state machine (§12.4) — safe to run concurrently with itself
Daily	<code>billing:process-subscription-renewals</code>	Guarded by Invoice uniqueness per subscription-period (§12.6)
Daily	<code>leave:accrue-monthly</code> (no-ops outside accrual date)	<code>leave_accruals</code> unique (<code>employee_id</code> , <code>period</code>) ledger (§15.1)
Daily	<code>loans:process-repayments</code>	Repayment schedule rows transition once, guarded by status check
Weekly	<code>webhooks:cleanup-stale-endpoints</code>	Read-only analysis + flag, not destructive
Monthly	<code>audit:partition-maintain</code>	Idempotent partition create/detach (§5.5)
Nightly	<code>notifications:cleanup-old</code>	Deletes read notifications past retention window — bounded, safe to re-run

Every scheduled command is wrapped in Laravel's `->onOneServer()` (when running multiple scheduler-capable instances) and `->withoutOverlapping()`, and failures are routed to the same Sentry/structured-log pipeline as request-time errors (Part 27–29) — a silently-failing scheduled job is one of the easiest production incidents to miss without this.

Part 37 — Scaling Strategy

37.1 Growth Table — What Scales Linearly and What Doesn't

Tenants	Employees (largest tenant)	What breaks first, and the mitigation already in this guide
10	100	Nothing — Phase 1 topology (§2.3) handles this comfortably on modest hardware
100	1,000	<code>employees</code> listing queries start to benefit from the <code>(tenant_id, company_id, status)</code> index (§5.3) already in place; still Phase 1
1,000	10,000	Payroll-run processing time per company grows linearly with employee count (the engine is $O(n)$ by design, §9), but a single company's run stays well under the queue-latency targets (§26.2); Postgres connection count starts to matter — move to PgBouncer connection pooling (Phase 2 signal)
10,000	100,000	A single company's payroll run (100k <code>GeneratePayslip</code> jobs, §13.6) is where job-dispatch overhead, not calculation time, becomes the bottleneck — batched job dispatch (<code>Bus::batch()</code>), chunked in groups of 500) replaces individual <code>dispatch()</code> calls per entry to avoid overwhelming Redis with 100k individual <code>LPUSH</code> calls in one request; read replica for dashboard/reporting queries (Phase 2)
100,000+ (across tenants)	1,000,000 (single company, hypothetical)	This is squarely Phase 3 territory (§2.3): payroll processing for a company this size is extracted into its own worker fleet with dedicated Postgres read capacity, sized and scaled independently of the shared API/notification workload — extraction justified by measured contention, not this table alone

Workloads that do not scale linearly, called out explicitly: compliance report generation (§11) scales with the *reporting period's* entry count, not the live employee count, so a company with high turnover has larger historical reports than its current headcount suggests; webhook delivery (§18.2) scales with the number of *subscribed events*, which for a tenant with several registered endpoints multiplies job count per triggering event, not per employee.

37.2 Stateless HTTP, Statefulness Isolated to Redis/Postgres

Every Laravel API instance is stateless — sessions (Redis), cache (Redis), and all durable data (Postgres) live outside the process, so horizontal scaling is “start another container,” no sticky-session requirement at the load balancer. PHP-FPM's process-per-request model makes this the natural default, unlike a long-lived Node event loop where in-memory state is an easy (and easy-to-misuse) shortcut — one place PHP's execution model removes a whole class of accidental-statefulness bug the original architecture had to actively guard against.

37.3 Connection Pooling, Pagination, Bulk Operations

PgBouncer (transaction-pooling mode) sits between the application and Postgres once instance count grows past what Postgres's own `max_connections` comfortably serves directly (each PHP-FPM worker holds its own connection for its request's duration). Every listing endpoint uses cursor pagination (`cursorPaginate()`) rather than offset pagination for any resource whose total count grows unboundedly (`payroll_entries`, `audit_logs`, `webhook_delivery_logs`) —

offset pagination's `OFFSET n` cost grows with `n`, cursor pagination's does not. Bulk writes (payroll entry insertion, §10.2) use `insert()` with chunked arrays, not `N` individual `Model::create()` calls. Large exports (Part 13's `exports` queue) use `LazyCollection` chunking (§5.4) to keep worker memory flat regardless of row count.

37.4 Distributed Locks, Rate Limiting, Idempotency at Scale

`Cache::lock()` (Redis-backed) is used for the payroll-run concurrency guard (§10.4) and any other “only one of these should run at a time across `N` instances” requirement (e.g., the outbox dispatcher's `FOR UPDATE SKIP LOCKED`, §13.7, which is itself a Postgres-native alternative to a Redis lock for that specific case). Rate limiting (login, public calculator, per-API-key throttling) uses Laravel's `RateLimiter` against the same shared Redis instance, so limits are enforced correctly regardless of which API instance handles a given request — a per-instance in-memory rate limiter would undercount attempts spread across a load-balanced fleet.

Part 38 — Production Hardening

38.1 Application

Strict `$fillable` (never `$guarded = []`) on every Eloquent model — mass-assignment vulnerabilities are exactly the kind of footgun §2.1 flagged Eloquent’s convenience-by-default nature enables, closed here by explicit discipline plus a PHPStan rule flagging any model without an explicit `$fillable` array. CSRF protection is Laravel’s default for the SPA-cookie auth path (§7.1); CORS is scoped tightly to the known frontend origin(s), never `*`, in `config/cors.php`. Every Form Request enforces `authorize()` — a Semgrep rule (Part 25) flags any state-changing controller method invoking a Form Request whose `authorize()` unconditionally returns `true`, which would otherwise silently disable the authorization check Part 7’s whole design assumes is present.

38.2 Infrastructure

TLS 1.2+ only, HSTS with a long max-age and `includeSubDomains` (§31.1), Nginx `client_max_body_size` capped appropriately per upload type, PHP-FPM `pm.max_children` tuned to the container’s memory ceiling to prevent an OOM kill from an under-tuned worker pool.

38.3 Secrets

Environment	Mechanism
Local	<code>.env</code> (never committed — enforced by Gitleaks, §25, and <code>.gitignore</code>)
CI	GitHub Actions encrypted secrets, scoped per environment
Staging/Production	A cloud-native secrets manager (AWS Secrets Manager or equivalent) — the application reads secrets at container start via a small init step that populates environment variables from the secrets manager, never bakes them into the image

Tradeoff considered: a dedicated tool like Vault or Doppler gives finer-grained access policies and audit logging on secret *reads*, not just storage, at the cost of running (or paying for) another piece of infrastructure. For PayrollFiti’s Phase 1–2 scale, a cloud-native secrets manager already integrated with the hosting platform’s IAM is the lower-overhead choice; Vault becomes the better tradeoff once secret-access-audit granularity (who read the Paystack key, and when) becomes a compliance requirement in its own right rather than a nice-to-have.

38.4 Environment Variable Strategy

Scope	Examples	Public?
Build-time (Next.js)	<code>NEXT_PUBLIC_API_URL</code> , <code>NEXT_PUBLIC_APP_NAME</code>	Yes — <code>NEXT_PUBLIC_*</code> variables are inlined into the client JS bundle at build time; anything prefixed this way must be treated as visible to anyone who opens devtools, never a secret
Runtime (Next.js server)	Sanity API token (read-only, for blog SSR/ISR)	No — server-only, never <code>NEXT_PUBLIC_*</code> -prefixed
Runtime (Laravel)	<code>APP_KEY</code> , <code>DB_PASSWORD</code> , <code>PAYSTACK_SECRET_KEY</code> , <code>MPESA_CONSUMER_SECRET</code>	No — never logged (§27.1), never returned in any API response, rotated via the secrets manager without an application code change

Part 39 — Developer Runbook

39.1 First-Time Setup

```
git clone <repo> && cd payrollfiti
cp backend/laravel/.env.example backend/laravel/.env
cp apps/web/.env.example apps/web/.env.local
docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d
docker compose exec api php artisan key:generate
docker compose exec api php artisan migrate --seed
open http://localhost:8080      # app, via Nginx
open http://localhost:8025     # Mailpit
open http://localhost:9001     # MinIO console
```

39.2 Day-to-Day

```
make test-backend    # php artisan test --parallel, inside the api container
make test-frontend   # pnpm --filter web test
docker compose logs -f horizon # watch queue processing during payroll-run development
docker compose exec api php artisan tinker # REPL against the local DB
docker compose exec api php artisan horizon:status
```

39.3 Debugging a Failed Payroll Run

1. Check Horizon's failed-jobs UI (or `php artisan queue:failed`) for the specific `GeneratePayslip / RunPayrollCommand`-related job and its exception.
2. Cross-reference the `correlation_id` from the failed job's log line (Part 27) against Sentry (Part 29) for full context — request payload, user, tenant.
3. If the failure is in the pure engine (Part 9), the fix is a unit test reproducing the exact `PayrollInput` first — the engine's purity means this is always reproducible offline, no database or queue involved.
4. Never manually `UPDATE payroll_entries` to “fix” a bad result — issue a correction run (Part 10 §10.3) so the audit trail stays intact.

39.4 Adding a Fourth Country

1. `app/Domain/Payroll/Engine/Rules/<Country>/` — implement `CountryRuleSet`, with unit tests mirroring §9.10's pattern.
2. Register in `RuleRegistry`'s constructor (§9.7).
3. `app/Domain/Compliance/Reports/<Country>/` — implement the country's statutory report classes (Part 11 §11.2).
4. Add the country to the public calculator's supported-country list (§10.5) and the frontend's country picker.
5. No change required anywhere else — the engine, the payroll API, the queue jobs, and the frontend calculator are all country-agnostic by design.

Part 40 — Production Readiness Checklist

Application

- ☐ Sanctum cookie + token auth both tested (Part 7)
- ☐ Gates + Policies cover every state-changing endpoint (Part 7 §7.6)
- ☐ Tenant isolation: app-layer scope + Postgres RLS both verified (Part 8)
- ☐ Rate limiting at Nginx and Laravel layers (Part 31, Part 37 §37.4)
- ☐ Every Form Request validates and authorizes (Part 38 §38.1)
- ☐ Payroll-run and payment idempotency verified under real concurrency (Part 9 §9.9, Part 10 §10.4, Part 12 §12.2)
- ☐ Audit logging active on every auditable model (Part 18 §18.3)

Payroll

- ☐ Payroll engine has zero framework dependencies, enforced by `deptrac` in CI (Part 9 §9.1)
- ☐ Every country ruleset versioned and resolved by effective date, never `now()` (Part 9 §9.7)
- ☐ Historical rule versions never mutated after a newer version ships (Part 9 §9.8)
- ☐ Concurrent duplicate payroll-run submissions produce one result (Part 10 §10.4)
- ☐ Corrections create new runs, never mutate historical entries (Part 10 §10.3)

Infrastructure

- ☐ Docker images built multi-stage, API and worker images separated (Part 32)
- ☐ Nginx TLS, security headers, and rate limiting configured (Part 31)
- ☐ `/health`, `/ready`, `/live` all distinguish liveness from dependency readiness (Part 25-equivalent health-check design, §4.1)
- ☐ Redis HA (Sentinel/cluster) configured before Phase 2 traffic (Part 37)
- ☐ PostgreSQL primary + replica, connection pooling in place at Phase 2 scale (Part 37 §37.3)
- ☐ Backups: WAL archiving active, restore drill scheduled and executed at least once (Part 36)

Observability

- ☐ Prometheus scraping `/metrics` (internal-only) with all Part 28 metrics present
- ☐ Grafana dashboards provisioned as code for Application, Payroll, Queues, Database, Redis, Payments
- ☐ Sentry active in staging/production, DSN-gated off locally (Part 29)
- ☐ Structured JSON logs with correlation IDs across HTTP → queue jobs (Part 27)
- ☐ Alerting thresholds defined with severities, avoiding alert fatigue (Part 28-adjacent — define `P0` – `P3` per the original brief's §45 before go-live)

Testing

- ☐ Unit (engine + components), Feature (HTTP), Integration (real Postgres/Redis/S3), Contract (OpenAPI), E2E (Playwright), Load (k6), Security (Part 25's five tools) all running in CI
- ☐

Tenant-isolation and concurrency tests tagged and run as a dedicated fast-feedback suite (Part 24 §24.3, §24.7)

Deployment

- CI: lint, static analysis, `deptrac`, tests, security scans, Docker build + Trivy scan (Part 34)
 - CD: staging auto-deploy, manual production approval gate, migration job separated from app containers (Part 35)
 - Expand/contract migration discipline documented and followed for every non-additive schema change (Part 35 §35.2)
 - Rollback path tested: previous image tag redeploy verified to work within the target RTO
 - Secrets never in source control, rotated via the secrets manager without code changes (Part 38 §38.3)
-

This guide preserves every functional requirement of the original PayrollFiti build guide while re-deriving its implementation around Laravel's idioms — most consequentially, adding PostgreSQL Row-Level Security as a second, database-level tenant-isolation layer (Part 8), moving statutory money arithmetic onto `bcmath` decimal precision (Part 9 §9.4), and replacing the original's manual JWT access/refresh session model with Sanctum's structurally simpler cookie-based multi-session design (Part 7 §7.5, §2.4). Every other architectural decision — the pure payroll core, versioned country rules, structural tenant isolation as a first-class requirement, idempotent payroll execution, and the modular-monolith-first scaling philosophy — is carried forward unchanged in substance, only in implementation mechanics.